

公告

昵称: ramlife 
园龄: 8年8个月
粉丝: 15
关注: 1
[+加关注](#)

搜索

[找找看](#)

常用链接

[我的随笔](#)
[我的评论](#)
[我的参与](#)
[最新评论](#)
[我的标签](#)

我的标签

[linux\(86\)](#)
[C/C++\(39\)](#)
[hardware\(24\)](#)
[QT\(21\)](#)
[android\(20\)](#)
[version\(19\)](#)
[vps\(14\)](#)
[network\(11\)](#)
[driver\(11\)](#)

[转] [Ozone] Ozone使用介绍-Trace功能

转自: <http://www.armbbs.cn/forum.php?mod=viewthread&tid=95867&highlight=Ozone>

承接上一个帖子:

Ozone使用介绍-基础功能

<http://armbbs.cn/forum.php?mod=viewthread&tid=95855&extra=page%3D1>

因为废话和贴图太多, 一个帖子不方便看, 所以分为了三个帖子, 本帖介绍Trace功能。

其实几个月前还是没有太关注Trace这个功能, 最近趁购买J-Trace的时候, 也顺道看了看Trace的一些相关介绍。

Trace其实分为很多种 (自己瞎划分的哈, 大家随便看看):

- 1、自己写printf, 根据自己的printf来进行跟踪
- 2、利用基于MCU支持的Jlink_RTT、USB CDC、TCP/IP、串口等种方式的流模式, 和一些内嵌入工程的接口调试代码, 来提供Trace记录, 如SystemView(基于Segger的RTT, 号称可以跟踪裸机不知道怎么做到的)、Tracealyzer (基于多种流模式)

System介绍: <https://www.segger.com/products/development-tools/systemview/>

win(10)
更多

随笔分类

android(12)
build(3)
C/C++(30)
deep_learning(3)
edit(9)
electron(5)
ffmpeg(1)
GTD(2)
hardware(11)
JAVA(4)
linux(64)
math(3)
network(7)
node.js(1)
opencv(2)
更多

随笔档案

2024年12月(1)
2024年11月(7)
2023年12月(1)
2023年2月(4)
2023年1月(11)
2022年12月(6)
2022年11月(1)
2022年10月(10)
2022年9月(12)
2022年8月(14)
2021年3月(3)
2021年2月(2)
2021年1月(4)
2020年12月(18)

SystemView



SystemView — System Verification Tool

- Minimally system intrusive
- Free tool. No license cost, no hidden fees
- RTOS task, resource, and API tracing
- Interrupt tracing for bare metal systems **without an RTOS**
- Continuous real-time recording and live analysis with J-Link and SEGGER RTT technology
- Live analysis of captured data - view responses to stimuli in real time without stopping the target
- SEGGER embOS, emNet, and emFile API call tracing as standard
- uC/OS-III, Micrium OS Kernel, and FreeRTOS instrumentation included
- Can be adapted to other RTOS using a fully documented API
- Works on any CPU

2020年11月(13)
更多

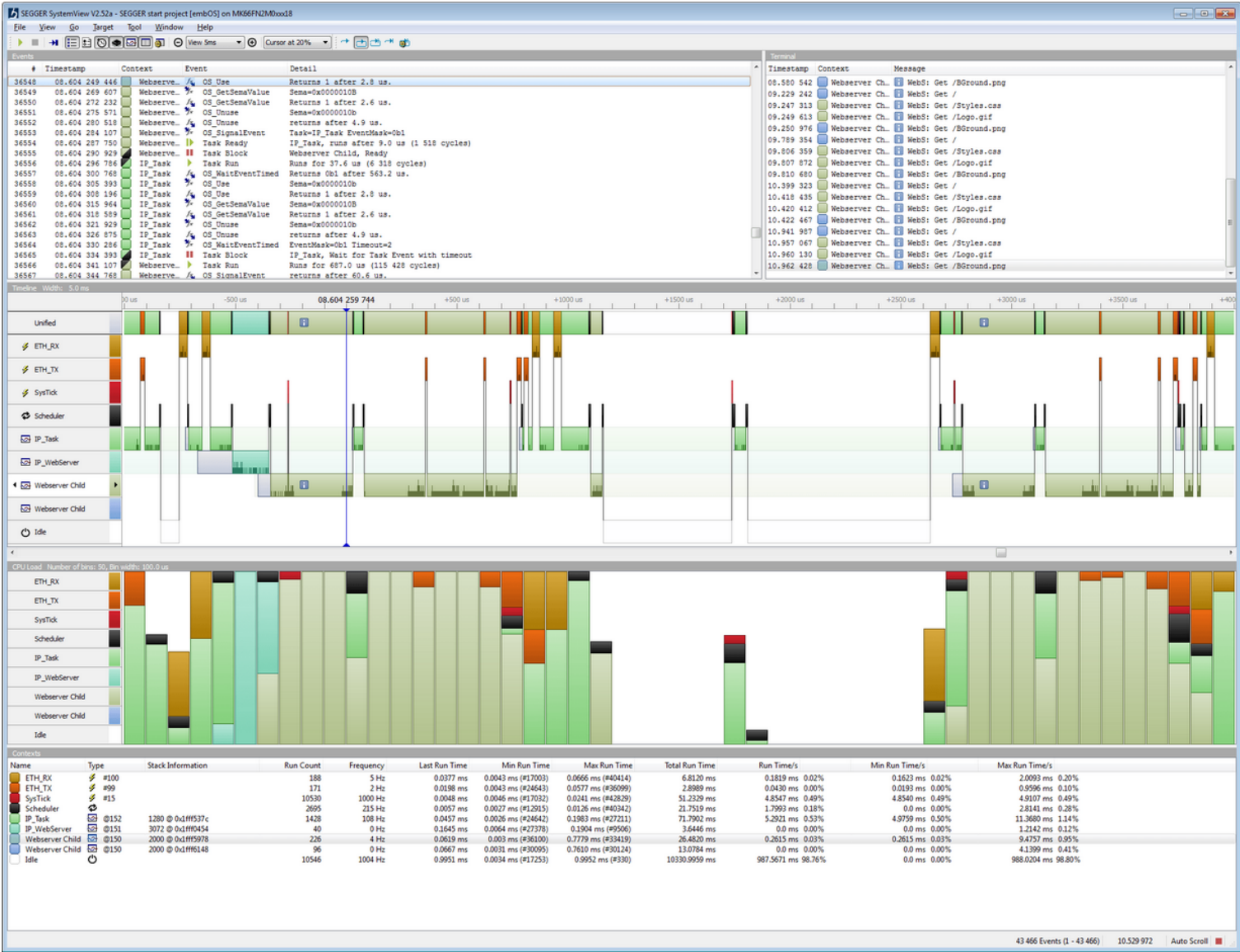
文章分类

android(5)
C/C++(15)
crack(9)
debug(6)
edit(1)
en(1)
GTD(1)
hardware(11)
JAVA(4)
jni(5)
linux(24)
network(3)
politics(5)
QT(5)
version(4)
更多

阅读排行榜

- 1. git 下载指定版本(14266)
- 2. I2C 读取总是 0xFF，但是 ACK 是正常的解决方法(11946)
- 3. qt 字符串 转换 hex(10551)
- 4. 光耦作为信号输入隔离时，输入 并联电阻，输出接斯密特反相器作用。(6836)
- 5. core_cm4.h(129): error: #35: # error directive: "Compiler generates FPU instructions for a device without an FPU (check __FPU_PRESENT)"(6734)

评论排行榜



Tracealyzer介绍: <https://percepio.com/tracealyzer/>

1. tensorflow lite c++(3)
2. stm32cubeide 代码提醒(2)
3. android 学习笔记(2)
4. ubuntu 20.04 安装 qtcreator(1)
5. 导出 立创 数据(1)

推荐排行榜

1. ssh 关闭密码登录(1)
2. MobileNetV2-SSDLite(1)
3. markdown 锚点, 分页, 表格内换行(1)
4. git 下载指定版本(1)
5. android 学习笔记(1)

最新评论

1. Re:导出 立创 数据

哥。、软件在哪儿下载

--LzxASDQWE

2. Re:lp_wizard 安装和使用

您好，可以发一下安装包和安装教程吗？谢谢

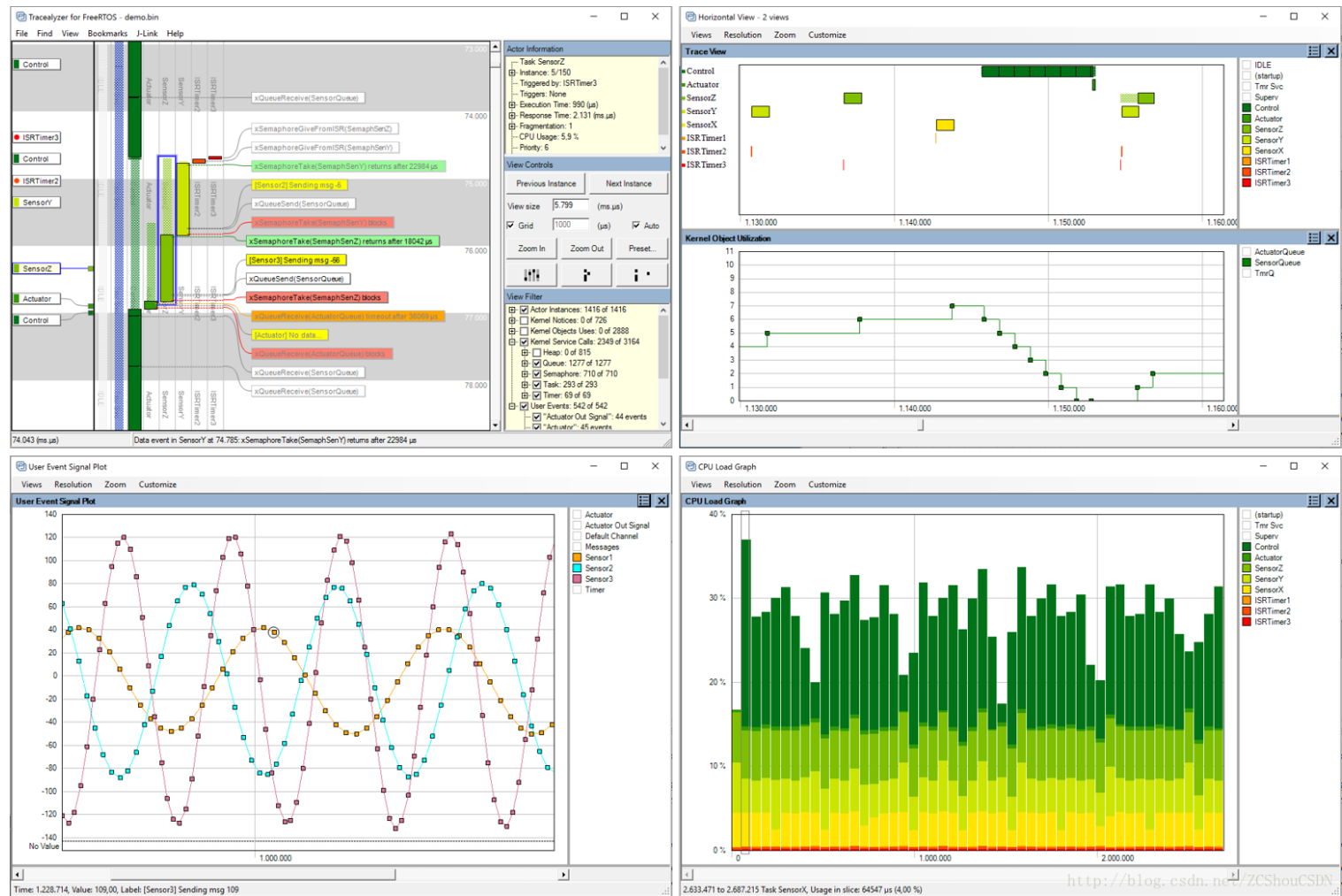
--1wll123

3. Re:[转]测试近 20 个版本的迅雷之后，告诉你哪个版本最好用

感谢分享，高人健康快乐！

--瓦雷利亚钢

4. Re:ubuntu 20.04 安装 qtcreator
install的s掉了: sudo apt
update sudo apt install gcc g++
+ build-essential mesa-



3、利用基于MCU硬件的Trace接口（比如Trace Pins、TraceBuffer、SWO）而实现的追踪功能。其中SWO（Serial Wire Viewer(SWV)）速度很低，但是占用接口很少；TraceBuffer（ETB算是）的上限会基于MCU本身的buffer大小；Trace Pins（ETM）只是负责实时上报出去，由跟踪调试器（J-Trace等）进行对应的协议转发或处理。

以下是Ozone手册的介绍：

common-dev...

--KALI_linux-chen

5. Re:[转] [Ozone] Ozone使用介绍-特殊功能

Ozone可以将某个反转的GPIO口像示波器或者keil的逻辑分析仪一样输出波形图吗

--杰瑞鼠

Trace Settings Dialog

Trace Source

Selects the trace data channel to be used:

Trace Source	Description
Trace Pins	Instruction Trace (ETM) data is read realtime-continuously from the target's trace pins and supplied to Ozone's Trace Windows. This option requires a J-Trace debug probe to be employed (see <i>Streaming Trace</i> on page 158).
Trace Buffer	Instruction Trace (ETM) data is read from the target's trace data buffer and supplied to Ozone's Trace Windows.
SWO	"Printf-type" textual application (ITM) data is read via the SWO channel and supplied to Ozone's <i>Terminal Window</i> on page 133.

For detailed information on ETM and ITM trace and how to set up your hardware and software accordingly, please consult the [J-Link User Guide](#).

Note

The simultaneous use of multiple trace data channels in Ozone is currently not supported.

STM32F4提示不支持Buffer的形式

ARM官网也会对自家硬件的ETM功能做对应介绍：（ST的手册这方面没有啥介绍）<http://www2.keil.com/coresight/>

CoreSight™ Technology

Arm Cortex-M processor-based devices use the Arm CoreSight technology which introduces powerful new **debug and trace** capabilities ([CoreSight Definitions](#) provides a glossary of terms).

Debug features:

- Run Control of the processor allowing you to start and stop programs
- Single Step one source or assembler line
- Set breakpoints while the processor is running
- Read/write memory contents and peripheral registers on-the-fly
- Program internal and external FLASH memory

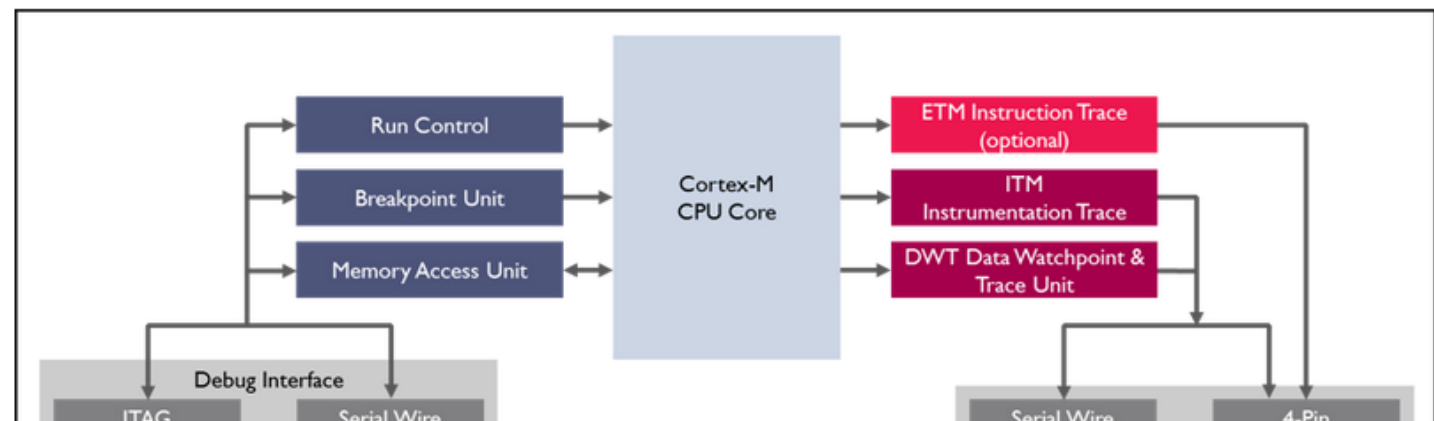
Trace features:

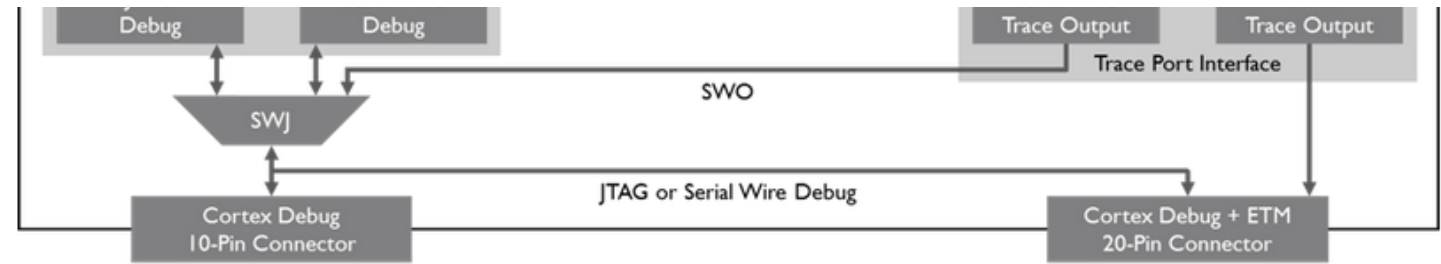
- [Serial Wire Viewer \(SWV\)](#) provides program counter (PC) sampling, data trace, event trace, and instrumentation trace information
- [Instruction \(ETM\) Trace](#) streamed directly to your PC enabling debugging of historical sequences, software profiling, and code coverage analysis

The CoreSight features are available via [JTAG](#) and [Serial Wire Debug](#) interfaces using standard low-cost target [connectors](#).

Quick Links

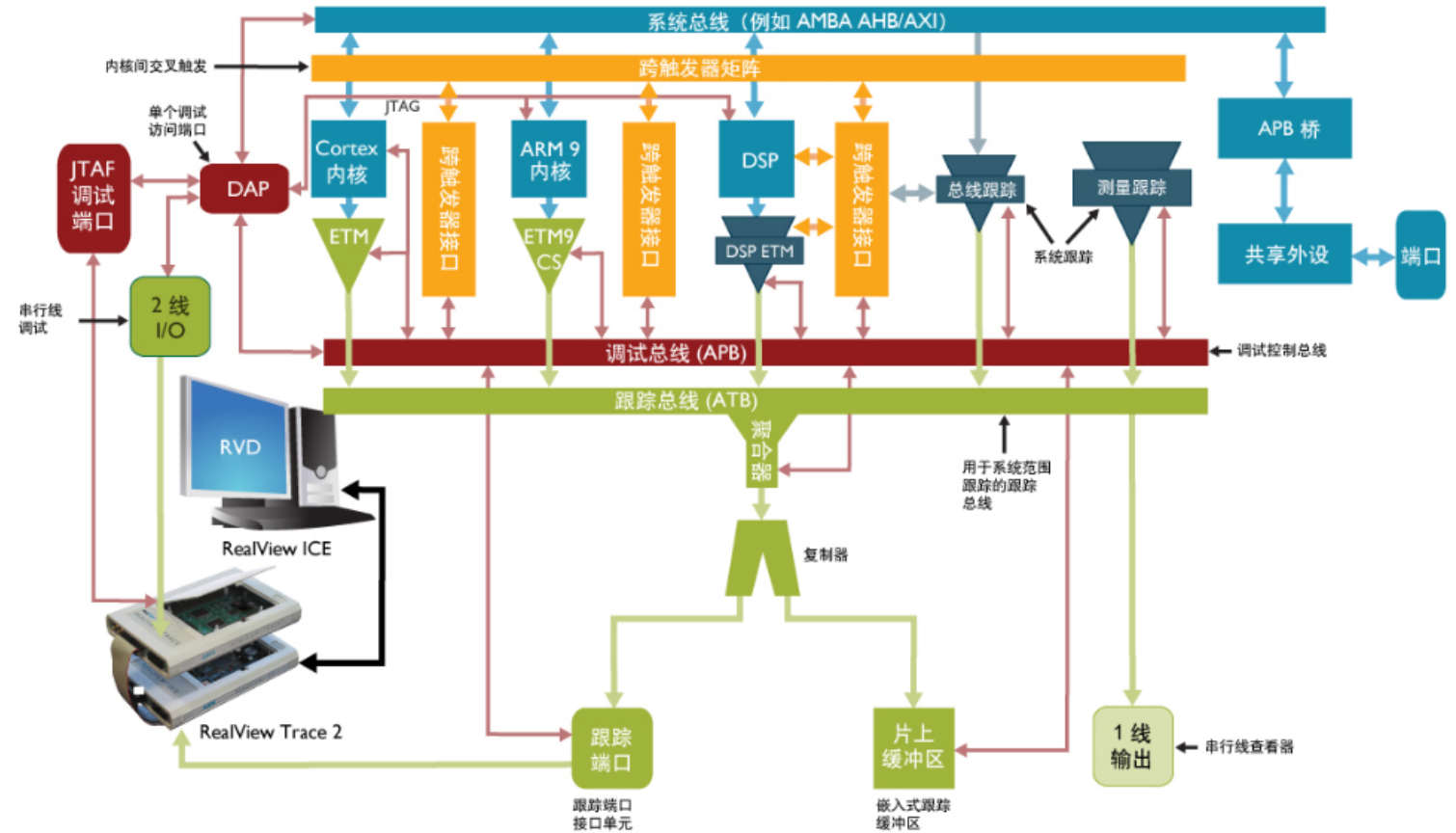
- [CoreSight Definitions](#)
- [CoreSight Connectors](#)
- [µVision Debugger](#)
- [µVision Debugger Knowledgebase](#)
- [ULINK Debug Adapters](#)





感觉其中2主要针对带有OS的目标程序进行追踪，主要说说3：

下图摘自<http://www.myir-tech.com/resource/510.asp>（米尔科技）



ETB，嵌入式跟踪缓存

CoreSightETB 是一个跟踪接收器，它可使用可配置大小的 RAM 为跟踪数据提供芯片上存储。

ETM，嵌入式跟踪宏单元

ETM宏单元为 ARM 微处理器提供实时指令跟踪和数据跟踪。跟踪软件工具使用 ETM 生成的信息重建全部或部分程序的执行情况。

Figure 1-1 shows a typical Cortex-M4 system in a System-on-Chip (SoC) that includes a CoreSight ETM-M4 macrocell.

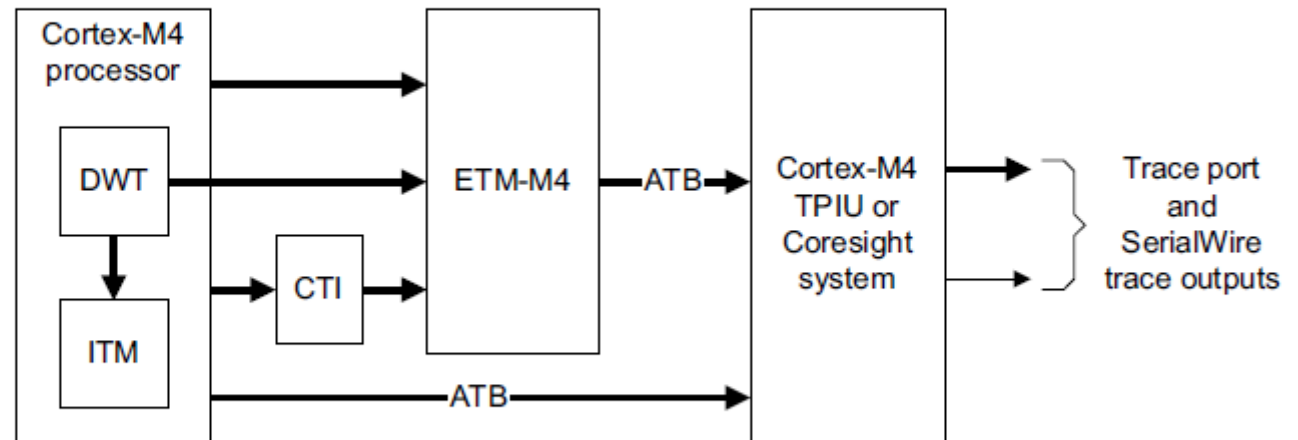


Figure 1-1 Cortex-M4 with ETM-M4 block diagram

CTI: Cross Trigger Interface ATB: Advanced Trace Bus

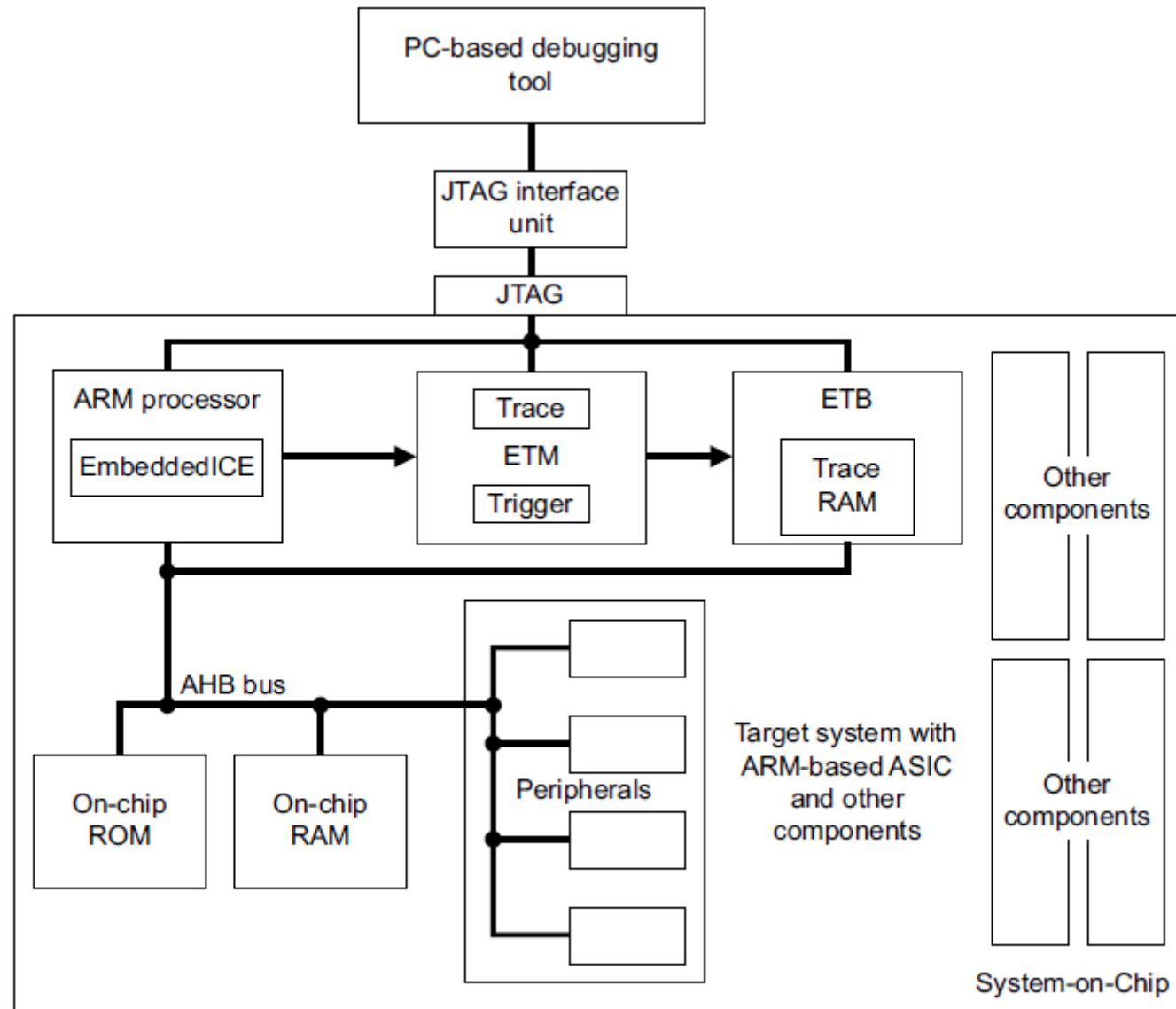


Figure 1-2 Example debugging environment with ETB

ETMv3.x

Figure 4-3 shows the structure of devices implementing ETMv3.x.

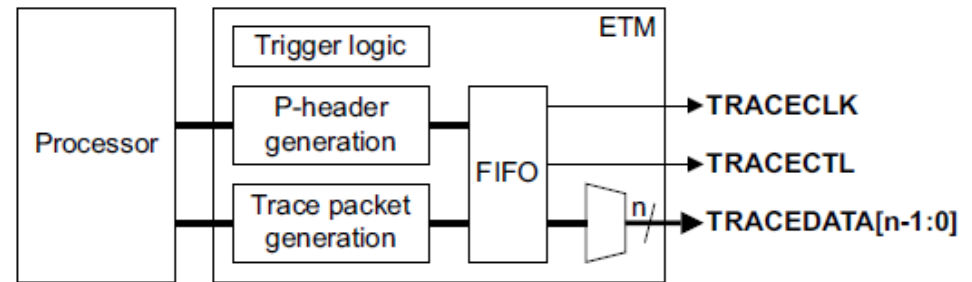


Figure 4-3 ETMv3.x structure

这么看的话，如果想要在STM32上实现硬件的Trace功能，需要的调试器本身就必须对接Trace接口。“大家都有便宜的”的Jlink是无法满足的，我之前购买的Stlink V3、DAPlink应该是都不行的。这个时候必须购买具备Trace功能（接口）的调试器了。在我决定购买Jlink Trace的时候，并没有比对过其他家具备Trace功能的调试器（好吧，就是不知道有.....，当时还以为IAR只做IDE工具，没有调试器的）。

我购买的：

德国的Segger：J-TracePro <https://www.segger.com/products/debug-trace-probes/>



[Contact Us](#)
[Forum](#)
[Wiki](#)
[Web Shop](#)
[Newsletter](#)
[RSS](#)

[Products](#)
[Downloads](#)
[Purchase](#)
[Support](#)
[About Us](#)


[Jobs](#)
[Blog](#)

Debug & Trace Probes



Market Leading Development Tools — J-Link and J-Trace PRO

The J-Link debug probes with their outstanding performance, robustness, and ease of use are the market leading debug probes today. The J-Trace PRO sets a benchmark for instruction tracing with its streaming trace function that enables unlimited tracing at full clock speed.

J-Trace PRO: The Leading Trace Solution



J-Trace PRO is an advanced debug probe that supports Arm's advanced tracing features of Arm Cortex cores.

Features

- Supports Streaming Trace (trace data is streamed to PC/debugger in real time, unlimited trace buffer)
- Has all the J-Link functionality
- Gigabit Ethernet interface
- SuperSpeed USB 3.0 interface (1.2 GBit/s)
- JTAG speed: 50 MHz
- Works with all currently available Cortex-M devices up to 150 MHz ETM trace clock (300 MHz CPU clock)
- Supports tracing on Cortex-M0/M0+/M1/M3/M4/M7 targets
- Free software updates, two years of support
- 64 MB on-board trace buffer for legacy one-shot tracing mode

后面搜了下，发现以下几家其实是都支持的。

瑞典的IAR: I-Jet Trace <https://www.iar.com/iar-embedded-workbench/add-ons-and-integrations/in-circuit-debugging-probes/>

Debugging and trace probes

[IAR Systems](#) > [IAR Embedded Workbench](#) > [Add-ons and integrations](#) >


Debugging and trace probes

Integrated tools simplify your development workflow. Our feature-rich in-circuit debugging probes enable powerful features in IAR Embedded Workbench.

I-jet Trace

I-jet

I-scope

Specifications



I-jet Trace for Arm Cortex-M

I-jet Trace for Arm Cortex-M provides extensive debugging and trace functionality. It delivers large trace memory capacities and high-speed communication via SuperSpeed USB 3.0. I-jet Trace is equipped with Embedded Trace Macrocell (ETM) trace, supporting all Cortex-M devices with ETM.



I-jet Trace for Arm Cortex-A/R/M

I-jet Trace for Arm Cortex-A/R/M provides extensive debugging and trace functionality for Arm Cortex devices equipped with the Arm CoreSight debug interface. It delivers large trace memory capacities and high-speed communication via SuperSpeed USB 3.0, and supports Embedded Trace Macrocell (ETM) trace as well as Program Trace Macrocell (PTM).

(算英国?) ARM (自家的宝宝) : ULINKpro <https://store.developer.arm.com/store/debug-probes/ulinkpro-debug-adapter#purchase>>

DEVELOPER STORE

DEBUG HARDWARE



Please note that we are currently unable to accept online orders from your region.

[See how to buy](#)

ULINKpro Debug Adapter

ULINKpro debug and trace unit programs, debugs and analyzes applications running on Arm Cortex-based devices using its full trace capability. The high-speed data and instruction trace is streamed directly to the PC providing detailed analysis of program behaviour, which is particularly helpful when certifying for functional safety. Streaming instruction trace



requires the target device to have ETM (Embedded Trace Macrocell).

Used in conjunction with the performance analysis within Arm Keil MDK, it identifies program bottlenecks, optimizes applications and isolates problems.

→ [See Product Comparison](#)

→ [See Features & Specifications](#)



德国LAUTERBACH（调试界的劳斯莱斯）：Cortex-M (ARMv6/7/832-bit) Debugger
<https://www.lauterbach.com/frames.html?home.html>

LAUTERBACH
DEVELOPMENT TOOLS

LEADING through Technology

[sitemap](#) | [print](#) | [mail](#) | [impressum](#)

<<

>>

International

日本語

Site

Chip

Supported Processor Architectures

Standard Controllers

ARM/XSCALE

☐ Cortex-A/R ARMv8
 ☐ Cortex-A ARMv7
 ☐ Cortex-R ARMv7
 ☐ Cortex-M
 ☐ ARM11
 ☐ ARM9
 ☐ ARM7
 ☐ JANUS
 ☐ XScale

Power Architecture

☐ Intel® Processors
 ☐ 78K0R/RL78
 ☐ AndesCore
 ☐ APS
 ☐ ARP32
 ☐ AVR8
 ☐ AVR32
 ☐ Beyond
 ☐ C166CBC
 ☐ 683xx
 ☐ ColdFire
 ☐ dsPIC
 ☐ eSi-RISC Core
 ☐ H8S/H8SX
 ☐ M32R

Cortex-M (ARMv6/7/8 32-bit) Debugger

▼ Debug and Trace Tools

▼ IDE - Integrated Development Environment

▼ Details and Configurations

Cortex-M

home > > > ARM/XSCALE

ARM

Cortex

Intelligent Processors by ARM

Introduction

The debugger supports the ARM Cortex™-M processor, which is used in highly deterministic real-time applications.

Highlights

- Easy high-level and assembler debugging
- Interface to all compilers
- RTOS awareness
- Display of internal and external peripherals at a logical level
- Flash programming
- Powerful script language
- Support for CoreSight components like Debug Access Port, Trace Funnel, Trace Port Interface Unit, Embedded Trace Buffer, Cross Trigger Interface, Cross Trigger Matrix, System Trace Port, Trace Memory Controller
- Real-time access to system memory and peripheral registers through Debug Access Port without halting the core
- Multicore debugging
- Multiprocessor debugging
- High-Speed download
- Support for

ADUCM3, ATSAM3, ATSAM4, ATSAM-C, ATSAM-D, ATSAM-D, ATSAM-E, ATSAM-G, ATSAM-L, ATSAM-R, ATSAM-S, ATSAM-V, EFM32 Gecko, F28M3x, FM3, FM4, Kinetis-E, Kinetis-EA, Kinetis-K, Kinetis-L, Kinetis-M, Kinetis-V, Kinetis-W, LPC800, LPC1100, LPC1200, LPC1300, LPC1500, LPC1700, LPC1800, LPC4000, LPC4300, LPC54000, MAX32600, MSP432, NRF51, NRF52, NUC, PSoC4, PSoC5LP, S32K, S6E, SimpleLink CC2x, SmartFusion, SmartFusion2, STM32L0, STM32L1, STM32L4, STM32F0, STM32F1, STM32F2, STM32F3, STM32F4, STM32F7, STM32W1, TLE984x, TLE986x, TLE987x, TM4C, TX03, Vybrid, XMC1000, XMC4000

其他的欢迎大家补充，支持Cortex M系列的我目前知道的就是这几个了。具体性能不敢对比，都是调试界的泰山北斗，算是神仙打架吧.....；每家支持跟踪的调试器，都有配套自己的跟踪工具（I-Jet和ULINKpro好像是集成进入IAR和Keil中了）

毕竟买的是J-Trace，上一张和Jlink的对比图，发现还是会大一圈的，如下图

J-Trace PRO

Cortex-A/R/M



www.segger.com

Before connecting J-Trace PRO Cortex-A/R/M, please visit the "Getting Started" page:

<https://www.segger.com/jtrace-pro-cortex-getting-started.html>

For support inquiries, please contact: support_jlink@segger.com

Bill of materials:

- J-Trace PRO Cortex-A/R/M
- Trace Reference Board
- 20-pin, 0.1" Target Ribbon Cable
- 20-pin, 0.05" Trace Ribbon Cable
- Ethernet Cable
- USB 3.0 Cable
- Mini USB Cable





其中Jlink&Trace的手册也给出了连接方法，和PCB的接口是对应的

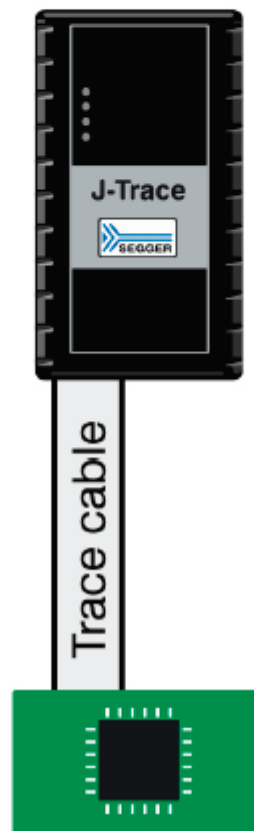
18.2.1 Connecting the target board

J-Trace connects to the target board via a 19-pin trace cable. Alternatively J-Trace can be connected with a 20-pin JTAG cable.

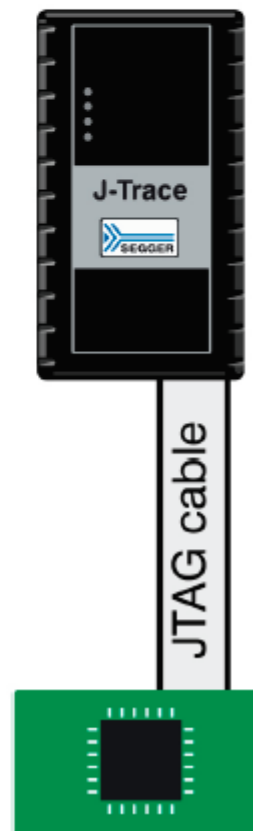
Note

Never connect trace cable and JTAG cable at the same time because this may lead to unstable debug and trace connections.

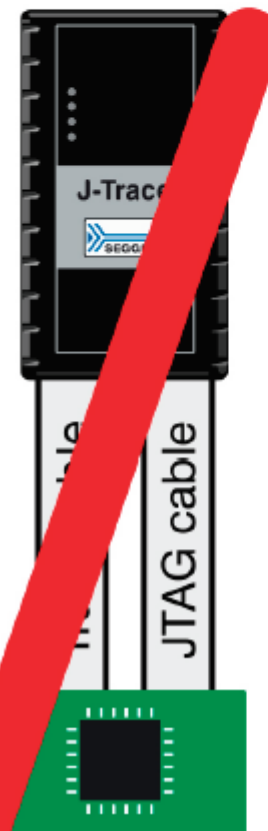
Debug + Trace



Debug only



wrong

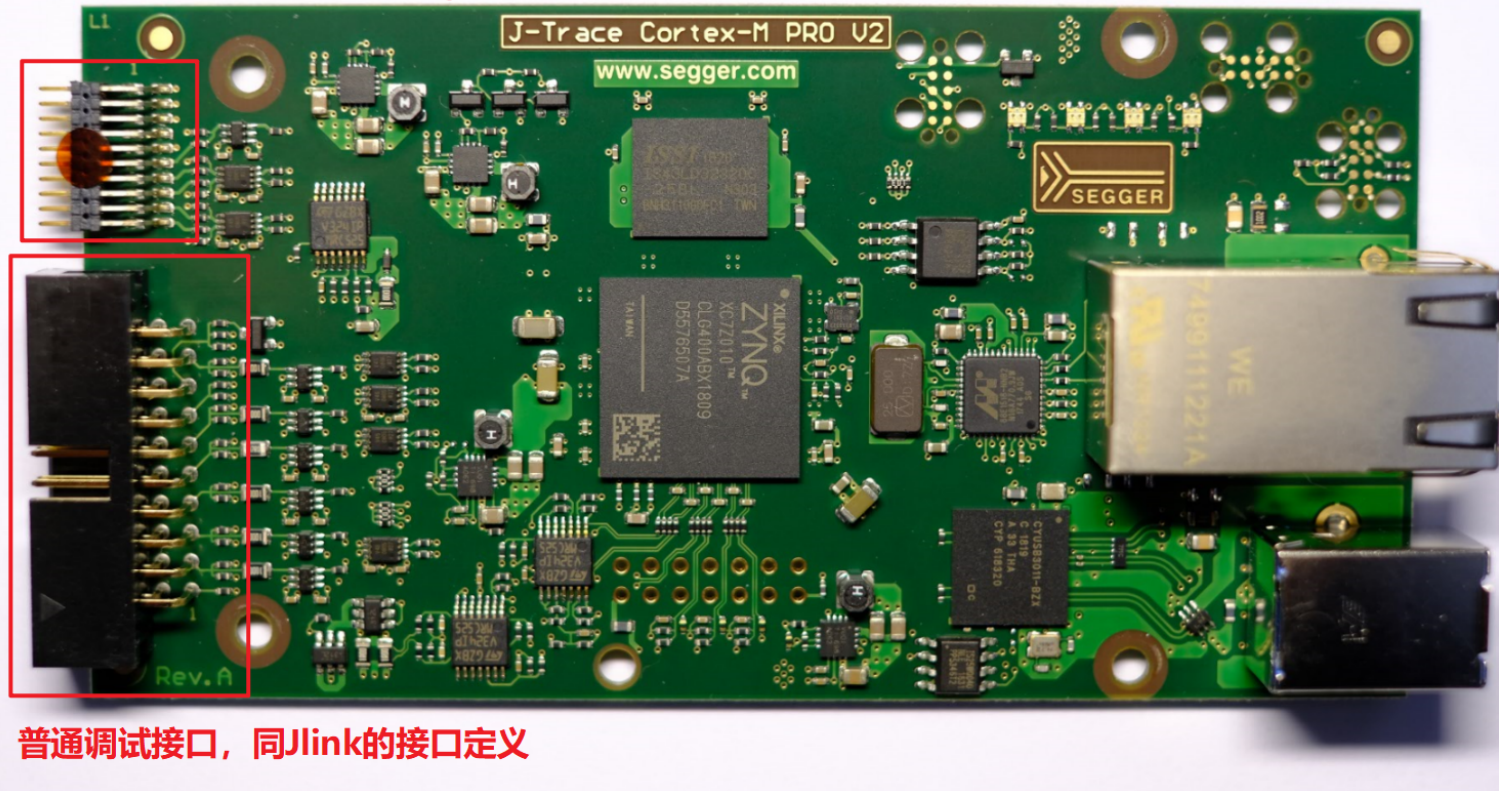


Target board

Target board

Target board

Trace接口 (包含SWD和Jtag功能)



后面的Trace都是基于Ozone进行演示的，如果有其他“小伙伴”有其它Trace工具的，欢迎借给我测试功能。

跟跟踪相关的三个窗口分别是Code Profile、InstructionTrace还有Timeline，这时候可以选择把这些窗口打开。

Ozone - The J-Link Debugger V2.70b - C:/Users/Ning/Desktop/J-T

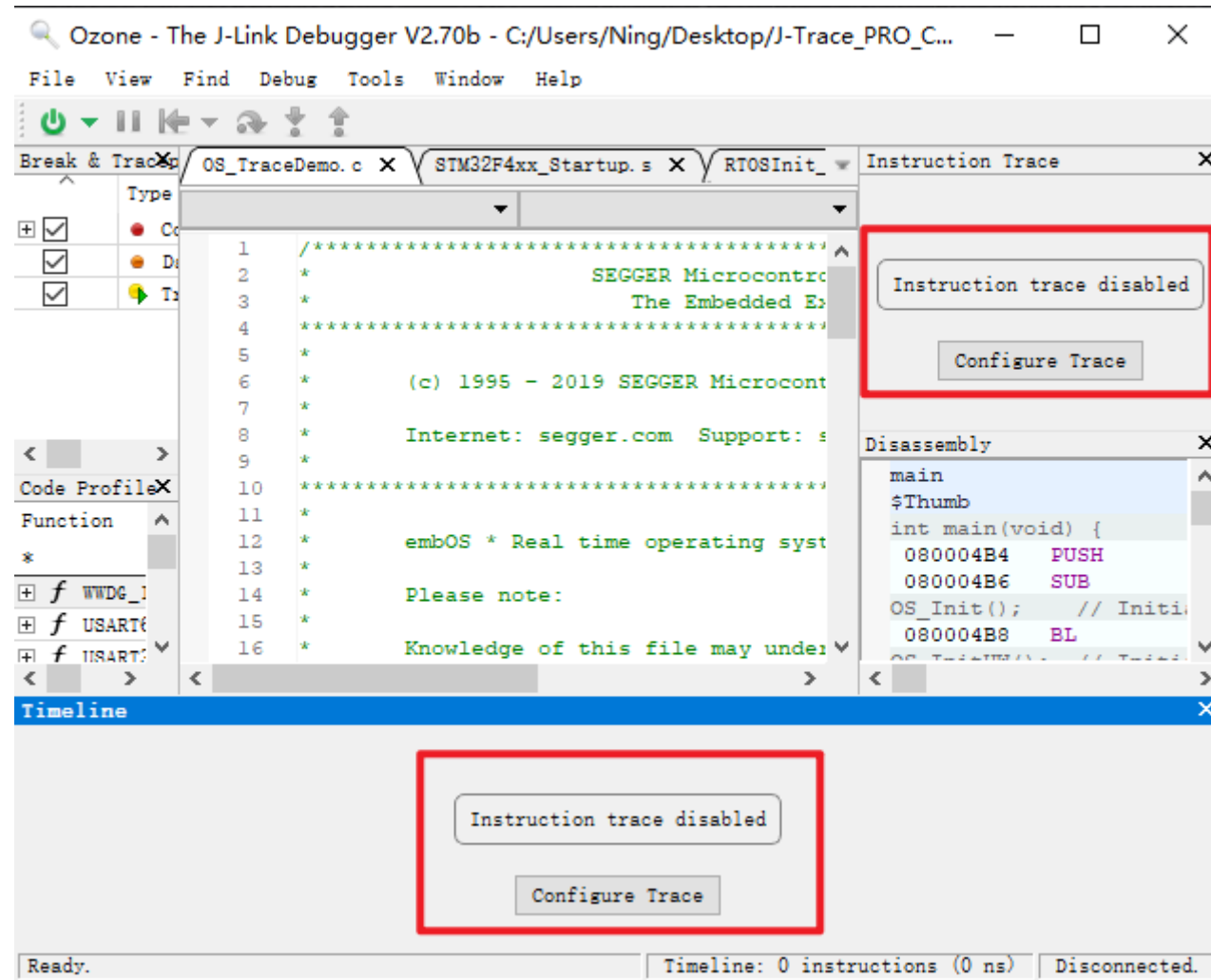
File View Find Debug Tools Window Help

The screenshot shows the 'View' menu in WinBox, which is used to toggle various toolbars and views. The menu is organized into several sections:

- General:**
 - Break & Tracepoints (Alt+Shift+B)
 - Call Stack (Alt+Shift+K)
 - Disassembly (Alt+Shift+D)
 - Global Data (Alt+Shift+G)
 - Local Data (Alt+Shift+L)
 - Memory
 - Registers
 - Terminal (Alt+Shift+T)
 - Watched Data (Alt+Shift+W)
- Advanced:**
 - Code Profile (Alt+Shift+P)
 - Data Graph (Alt+Shift+Y)
 - Instruction Trace
 - Power Graph (Alt+Shift+A)
 - Timeline (Alt+Shift+N)
 - _mbOS (Alt+Shift+O)
- Static:**
 - Call Graph (Alt+Shift+H)
 - Find Results (Alt+Shift+E)
 - Functions (Alt+Shift+F)
 - Memory Usage (Alt+Shift+S)
 - Source Files (Alt+Shift+U)
- Information:**
 - Console (Alt+Shift+C)
 - J-Link Control Panel (Alt+Shift+J)
 - Toolbars
 - Enter Full Screen (Alt+Shift+Return)

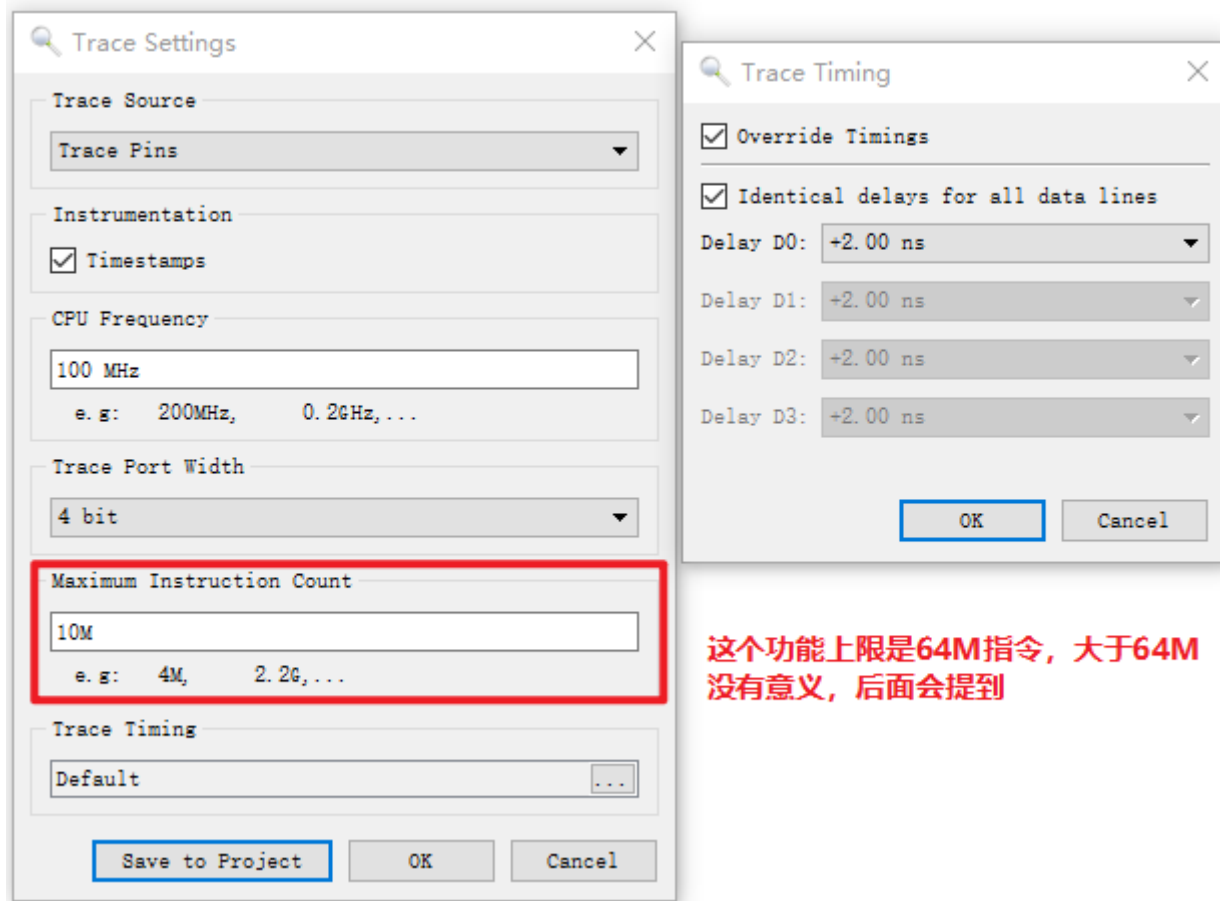
The 'Instruction Trace' and 'Timeline' items in the 'Advanced' section are highlighted with red boxes in the original image.

Trace需要进行配置才可以，因为上一偏帖子是关闭了Trace的，所以这时候在具备Trace的界面会提示Trace已经被禁止了，需要手动打开



配置有三种：

1、Trace Pins配置



其中时间标签（TimeStamps）可以选择勾选，官方手册上面写道，如果不勾选时间标签的话，还可以进一步提升跟踪性能。这个也好理解，毕竟都需要通过Trace的硬件接口（ARM的ETM协议）上报出来，减少了TimeStamps的参数（不再占传输带宽了），理应提高跟踪性能。

Note

When instruction timestamps are not required, the option should be disabled to enhance the overall tracing performance.

CPU时钟频率设置，我的理解是为了根据指令条目综合得到Timeline的时间轴信息的，毕竟Trace回来的指令只能编号，目标板也不可能提供时间信息，即使提供也太占传输带宽了。

TracePort 带宽的话，我觉得如果是目标板设计时，要么就会留出所有的端口（4个），要么一个也不会留，没道理项目上设计预留

Trace接口，还只给出了1或2个吧.....Segger提供的测试板是4个，所以选择4bit。
最大指令计数上限，这个地方目前Ozone的上限是64M，设置大于64M没有意义。这个后面再Timeline窗口会提到。

Trace-Time是个时间修正设置。根据Ozone的跟踪精度介绍，我的理解是在这里可以对跟踪的数据进行延迟的修正。对我意义不大，没有那么高的精度要求。

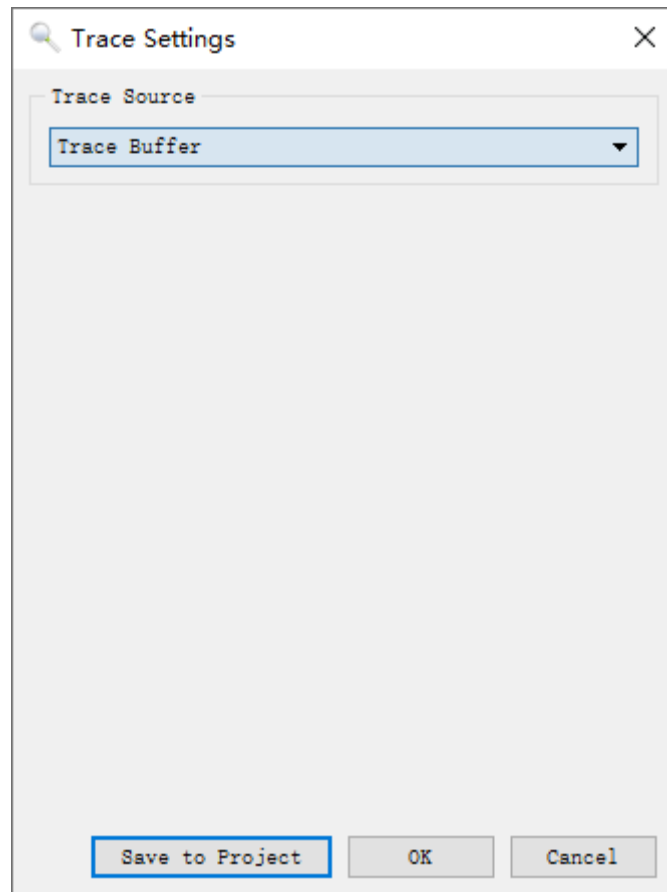
Tracepoint Imprecision



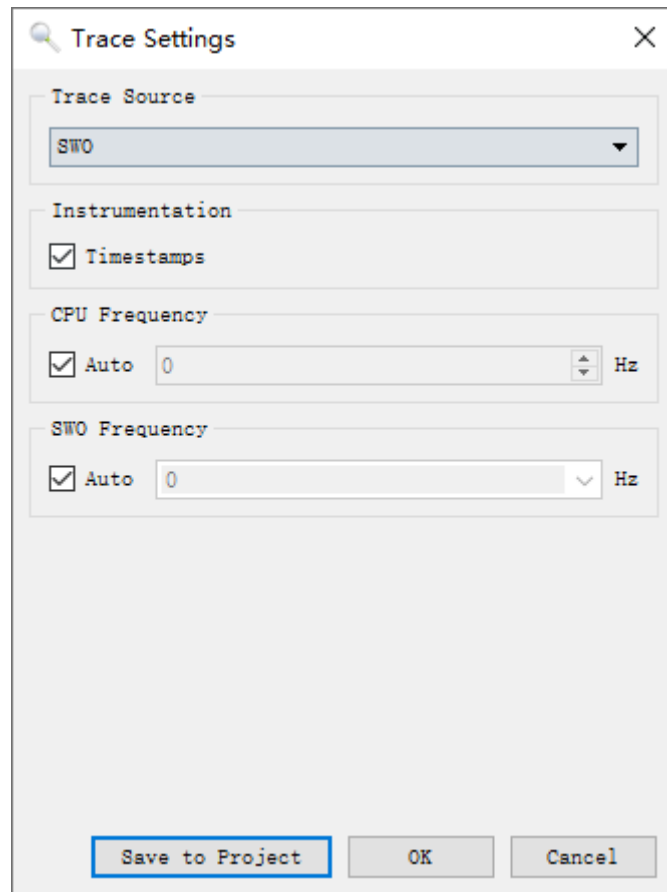
An MCU possibly commands its tracepoints hardware unit asynchronously to its instruction execution unit. This means that trace data capture may be started and stopped a few cycles after the affiliated instruction has been fetched for execution.

会有一定的延迟！MCU内的追踪单元会和指令执行单元不同步，慢几个周期

2、Trace Buffer: STM32F4不支持，不知道怎么测试



3、SWO：本次没有测试，毕竟已经有Trace Pin了



The image shows a 'Trace Settings' dialog box with a search icon and a close button (X) in the title bar. It contains four sections: 'Trace Source' with a dropdown menu set to 'SWO'; 'Instrumentation' with a checked checkbox for 'Timestamps'; 'CPU Frequency' with a checked checkbox for 'Auto' and a numeric field set to '0' followed by 'Hz'; and 'SWO Frequency' with a checked checkbox for 'Auto' and a numeric field set to '0' followed by 'Hz'. At the bottom are three buttons: 'Save to Project' (highlighted with a blue border), 'OK', and 'Cancel'.

Trace Settings

Trace Source

SWO

Instrumentation

☒ Timestamps

CPU Frequency

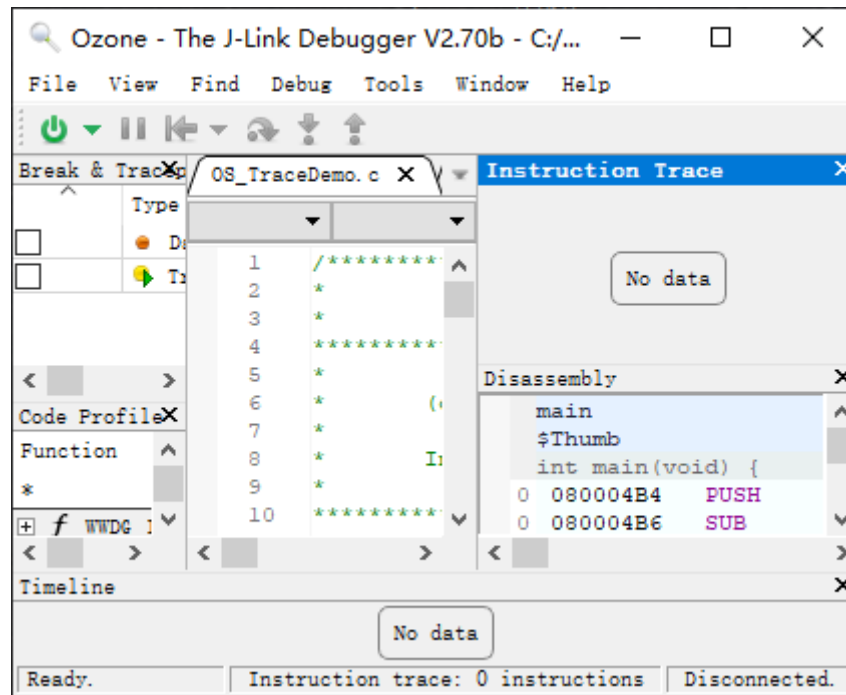
☒ Auto 0 Hz

SWO Frequency

☒ Auto 0 Hz

Save to Project OK Cancel

当正确设置完毕后，就可以进行追踪了，因为还没有开始RUN起来，所以追踪的窗口还是显示No Data



点开RUN的时候，程序和用其他IDE一样，已经

有个细节需要注意，Segger是有一套自己的（STM32F4xx_Startup.s）启动文件的，虽然套路和我们常用的ST的一样，都是从Reset_Handler启动，进行初始化的，两者的写法略有不同，不过后面都是调用system_stm32f4xx.c（这个都是ST提供的了）文件。移植不同IDE工程的时候，这个地方也是需要一些改动的（比如从Keil移植到ES）。

```

1;***** (C) COPYRIGHT 2013 STMicroelectronics *****
2;* File Name      : startup_stm32f40_4lxxx.s
3;* Author         : MCD Application Team
4;* Version        : V1.3.0
5;* Date           : 08-November-2013
6;* Description    : STM32F40xxx/4lxxx devices vector table for MDK-ARM toolchain.
7;*               : This module performs:
8;*               : - Set the initial SP
9;*               : - Set the initial PC == Reset_Handler
10;*              : - Set the vector table entries with the exceptions ISR address
11;*              : - Configure the system clock and the external SRAM mounted on
12;*               :   STM324xG-EVAL board to be used as data memory (optional,
13;*               :   to be enabled by user)
14;*              : - Branches to __main in the C library (which eventually
15;*               :   calls main()).
16;*               : After Reset the CortexM4 processor is in Thread mode,
17;*               : priority is Privileged, and the Stack is set to Main.
18;* <<< Use Configuration Wizard in Context Menu >>>
19;*****
20;
21; Licensed under MCD-ST Liberty SW License Agreement V2, (the "License");
22; You may not use this file except in compliance with the License.
23; You may obtain a copy of the License at:
24;
25; http://www.st.com/software\_license\_agreement\_liberty\_v2
26;
27; Unless required by applicable law or agreed to in writing, software
28; distributed under the License is distributed on an "AS IS" BASIS,
29; WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
30; See the License for the specific language governing permissions and
31; limitations under the License.
32;
33;*****

```

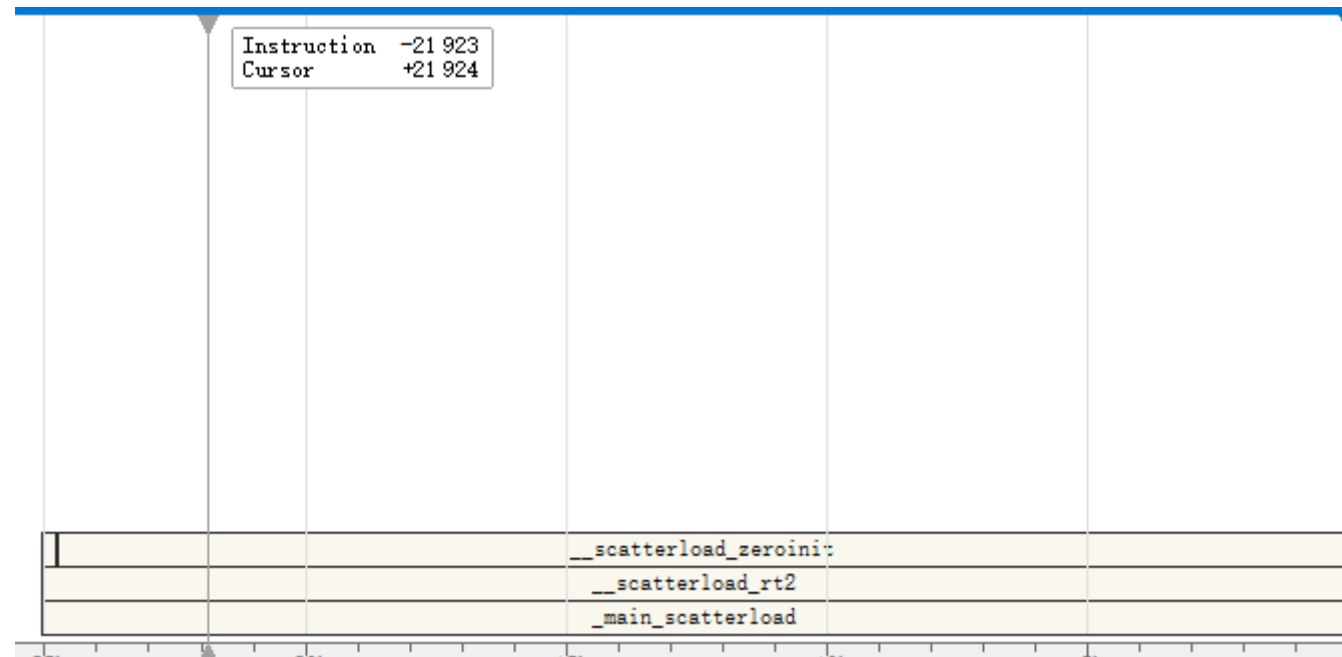
```

1  /*****
2  *                               SEGGER Microcontroller GmbH & Co. KG                               *
3  *                               Solutions for real time microcontroller applications                       *
4  *                               *****/
5  *
6  *                               (c) 2015 SEGGER Microcontroller GmbH & Co. KG                               *
7  *
8  *                               Internet: www.segger.com   Support: support@segger.com   *
9  *                               *****/
10 *****/
11
12 /*****
13 *                               Preprocessor Definitions                                           *
14 *                               -----*
15 * NO_FPU_ENABLE*
16 *
17 *   If defined, FPU will not be enabled.*
18 *
19 * NO_STACK_INIT*
20 *
21 *   If defined, the stack pointer will not be initialised.*
22 *
23 * NO_SYSTEM_INIT*
24 *
25 *   If defined, the SystemInit() function will not be called. By default*
26 *   SystemInit() is called after reset to enable the clocks and memories to*
27 *   be initialised prior to any C startup initialisation.*
28 *
29 * NO_VTOR_CONFIG*
30 *
31 *   If defined, the vector table offset register will not be configured.*
32 *
33 * MEMORY_INIT*
34 *
35 *   If defined, the MemoryInit() function will be called. By default*
36 *   MemoryInit() is called after SystemInit() to enable an external memory*
37 *   controller.*
38 *
39 * STACK_INIT_VAL*
40 *
41 *   If defined, specifies the initial stack pointer value. If undefined,*
42 *   the stack pointer will be initialised to point to the end of the*
43 *   RAM segment.*
44 *
45 * VECTORS_IN_RAM*
46 *
47 *   If defined, the exception vectors will be copied from Flash to RAM.*
48 *
49 *****/

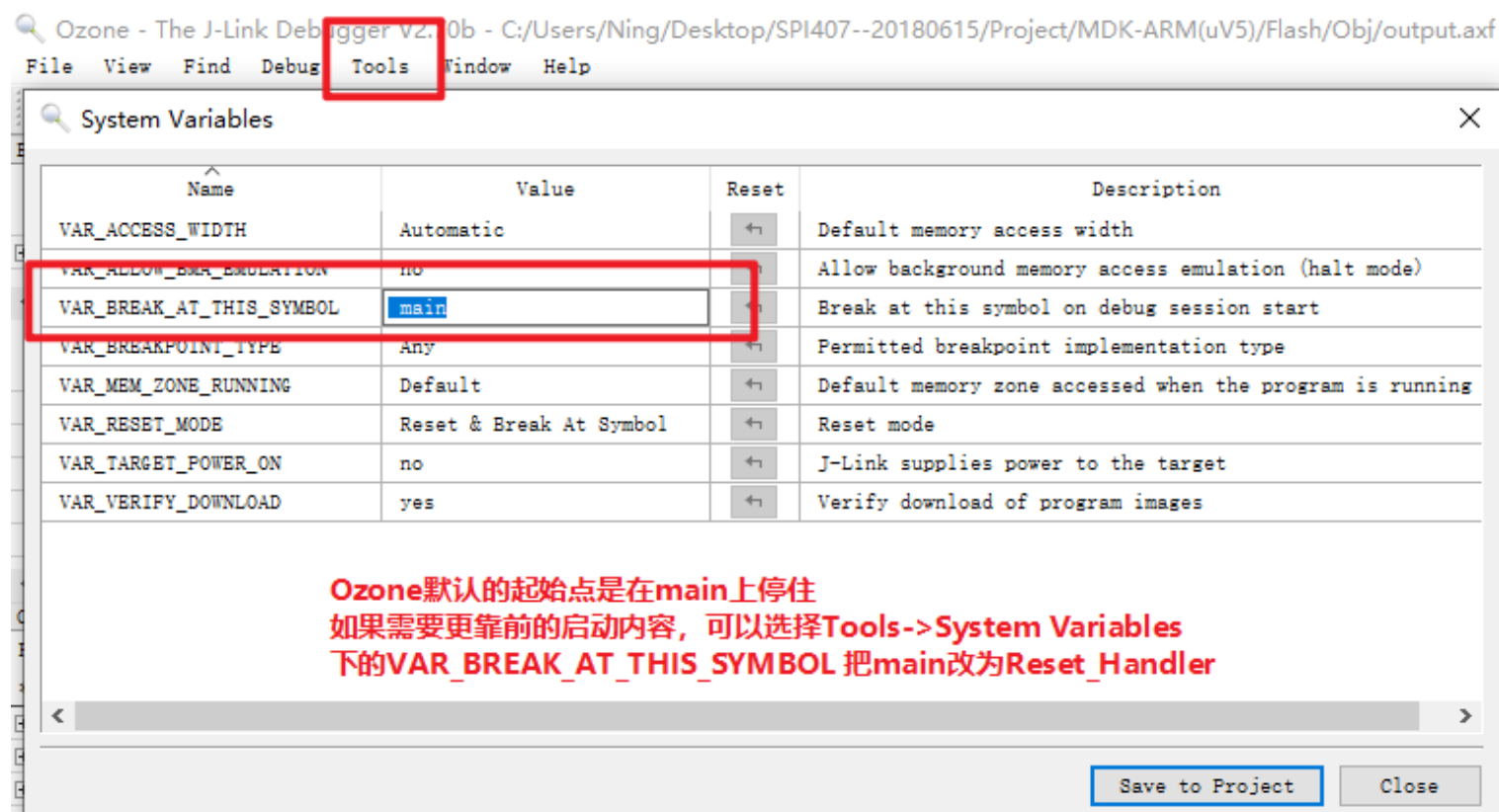
```

[illegible]

但是如果直接使用Keil工程包导入的使用ST自带的启动文件的话，Ozone并不能正确识别启动文件的操作。



Ozone的起始点是可以设置的



先上个全局的动态图看一下吧（这个图我指令跟踪上限是10M）

Ozone - The J-Link Debugger V2.70b - C:/Users/Ning/Desktop/J-Trace_PRO_CortexM_Tutorial/SEGGER_CortexM_Trace_Reference_Board.jdebug

File View Find Debug Tools Window Help

Break & Tracepoints

Type	Location	Extras
Data	2000 0F54	Write, 4 Bytes
Trace Start	RTOSInit_STT	

Code Profile

Function	Source Coverage	Inst. Coverage	Run Count	Fetch Count	Load
* f_SendPacket	0.0% (0/37)	0.0% (0/190)	0	0	0.00%
* f_start	0.0% (0/31)	0.0% (0/31)	0	0	0.00%
* f_SSG68R_RTT_ReadNoLock	0.0% (0/29)	0.0% (0/122)	0	0	0.00%
* f_SSG68R_RTT_WriteShipNoLock	0.0% (0/28)	0.0% (0/136)	0	0	0.00%
* f_HandleIncomingPacket	0.0% (0/27)	0.0% (0/79)	0	0	0.00%
* f_HardFaultHandler	0.0% (0/26)	0.0% (0/106)	0	0	0.00%
* f_SetSysClock	0.0% (0/23)	0.0% (0/96)	0	0	0.00%
* f_SystemCoreClockUpdate	0.0% (0/21)	0.0% (0/97)	0	0	0.00%
* f_SSG68R_SYSVIEW_Start	0.0% (0/21)	0.0% (0/161)	0	0	0.00%
* f_TLMMEM_SendChar	0.0% (0/21)	0.0% (0/63)	0	0	0.00%
* f_Fill7Buf	0.0% (0/20)	0.0% (0/44)	0	0	0.00%
* f_JoinInit	0.0% (0/20)	0.0% (0/62)	0	0	0.00%
* f_TLMMEM_Process	0.0% (0/19)	0.0% (0/62)	0	0	0.00%
* f_EncodeStr	0.0% (0/17)	0.0% (0/72)	0	0	0.00%
* f_SSG68R_SYSVIEW_SendModule	0.0% (0/16)	0.0% (0/113)	0	0	0.00%
* f_SSG68R_RTT_ConfigUpBuffer	0.0% (0/16)	0.0% (0/96)	0	0	0.00%
* f_Rsrc_Handler	0.0% (0/16)	0.0% (0/16)	0	0	0.00%
* f_SSG68R_SYSVIEW_SendTaskInfo	0.0% (0/16)	0.0% (0/209)	0	0	0.00%
* f_TrySendOverflowPacket	0.0% (0/14)	0.0% (0/103)	0	0	0.00%
* f_OS_InitHW	0.0% (0/14)	0.0% (0/59)	0	0	0.00%
* f_OS_InitHW	0.0% (0/13)	0.0% (0/81)	0	0	0.00%
* f_SSG68R_init_copy	0.0% (0/13)	0.0% (0/13)	0	0	0.00%
* f_SSG68R_SYSVIEW_SetSysDesc	0.0% (0/13)	0.0% (0/139)	0	0	0.00%
* f_SystemInit	0.0% (0/11)	0.0% (0/37)	0	0	0.00%
* f_SSG68R_SYSVIEW_RecordU32x3	0.0% (0/11)	0.0% (0/160)	0	0	0.00%
* f_SSG68R_SYSVIEW_Init	0.0% (0/11)	0.0% (0/45)	0	0	0.00%
* f_Send	0.0% (0/11)	0.0% (0/36)	0	0	0.00%
* f_cbSendTaskInfo	0.0% (0/11)	0.0% (0/30)	0	0	0.00%
* f_SSG68R_init_sero	0.0% (0/11)	0.0% (0/11)	0	0	0.00%
* f_SSG68R_SYSVIEW_RecordU32x4	0.0% (0/10)	0.0% (0/123)	0	0	0.00%
* f_SysTick_Config	0.0% (0/9)	0.0% (0/106)	0	0	0.00%
* f_SSG68R_SYSVIEW_RecordU32x3	0.0% (0/9)	0.0% (0/83)	0	0	0.00%
* f_SSG68R_SYSVIEW_OnTaskStopRe	0.0% (0/9)	0.0% (0/61)	0	0	0.00%
* f_main	0.0% (0/9)	0.0% (0/89)	0	0	0.00%
* f_BSP_Init	0.0% (0/9)	0.0% (0/56)	0	0	0.00%
* f_Receive	0.0% (0/9)	0.0% (0/25)	0	0	0.00%

Timeline

OS_TraceDemo.c

```
24 * OS version: 5.06
25 *
26 *
27 *
28 *
29 File : OS_StartLEDLink.c
30 Purpose : embOS sample program running two simple tasks, each toggling
31 a LED of the target hardware (as configured in BSP.c).
32 */
33
34 #include "RTOS.h"
35 #include "BSP.h"
36
37 static OS_STACKPTR int StackHP[128], StackLP[128]; // Task stacks
38 static OS_TASK TCBHP, TCBLP; // Task control block
39 static int _Cnt1;
40 static int _Cnt2;
41
42 #static void HPTask(void) {
43     while (1) {
44         BSP_ToggleLED(0);
45         _Cnt1++;
46         if (_Cnt1 == 100) {
47             _Cnt1 = 0;
48         }
49         OS_TASK_Delay(50);
50     }
51 }
52
53 #static void LPTask(void) {
54     while (1) {
55         BSP_ToggleLED(1);
56         _Cnt2++;
57         if (_Cnt2 == 100) {
58             _Cnt2 = 0;
59         }
60         OS_TASK_Delay(100);
61     }
62 }
63
64 //*****
65 *
66 * main()
67 */
68 #int main(void) {
69     OS_Init(); // Initialize embOS
70     OS_InitHW(); // Initialize required hardware
71     BSP_Init(); // Initialize LED ports
72     OS_TASK_CREATE(<TCBHP, "HP Task", 100, HPTask, StackHP);
73     OS_TASK_CREATE(<TCBLP, "LP Task", 50, LPTask, StackLP);
74     OS_Start(); // Start embOS
75     return 0;
76 }
77
78 //***** End of file *****
79
```

Disassembly

```
main
3Thumb
int main(void) {
0 08000484 PUSH R3, SP, #20
0 0800048B BL OS_Init_VFP ; 0x080003C8
0 0800048C BL OS_InitHW ; 0x08000CF4
0 0800048D BL BSP_Init ; 0x080005A0
0 0800048E BL OS_TASK_CREATE(<TCBHP, "HP Task", 100, HPTask, StackHP);
0 0800048F MOVW R3, #2
0 08000490 STR R3, [SP, #0]
0 08000491 MOVW R3, #0x200
0 08000492 STR R3, [SP, #4]
0 08000493 LDR R3, *StackHP ; [0x0800054C] = 0;
0 08000494 STR R3, [SP]
0 08000495 LDR R3, #0x00000000 ; [0x08000550]
0 08000496 MOVW R2, #0x4
0 08000497 LDR R1, #0x00000000 ; [0x08000554]
0 08000498 LDR R0, *TCBHP ; [0x08000558] = 0;
0 08000499 BL OS_TASK_Create_DP ; 0x08000DC4
0 0800049A BL OS_TASK_CREATE(<TCBLP, "LP Task", 50, LPTask, StackLP);
0 0800049B MOVW R3, #2
0 0800049C STR R3, [SP, #8]
}
```

1、Timeline时间线窗口：

当打开Trace功能，同时开始运行程序，Ozone会根据J-Trace不断通过接口协议（USB3.0 或者 千兆网线）上报的ETM数据进行记录。根据上报的ETM记录，可以重建整个时间线的指令执行情况。横轴指令数目或时间（根据指令计数和设置的MCU主频得导的时间），纵轴是函数之间的调用关系，不需要使用RTOS，裸跑也能实现。

4.21.8 Instruction Ticks

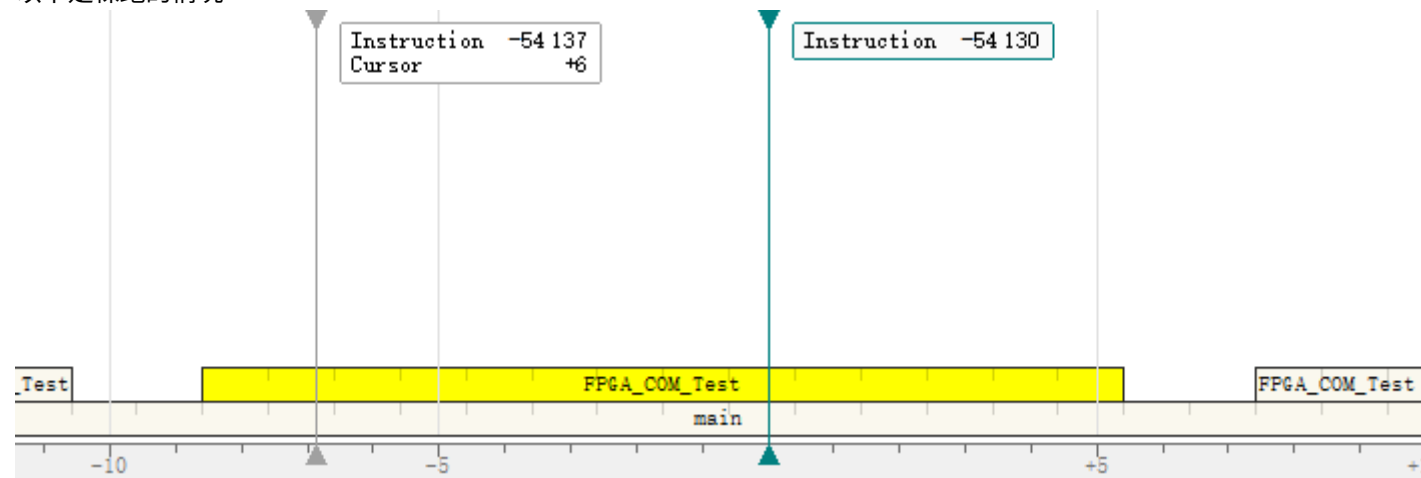
-400 ns

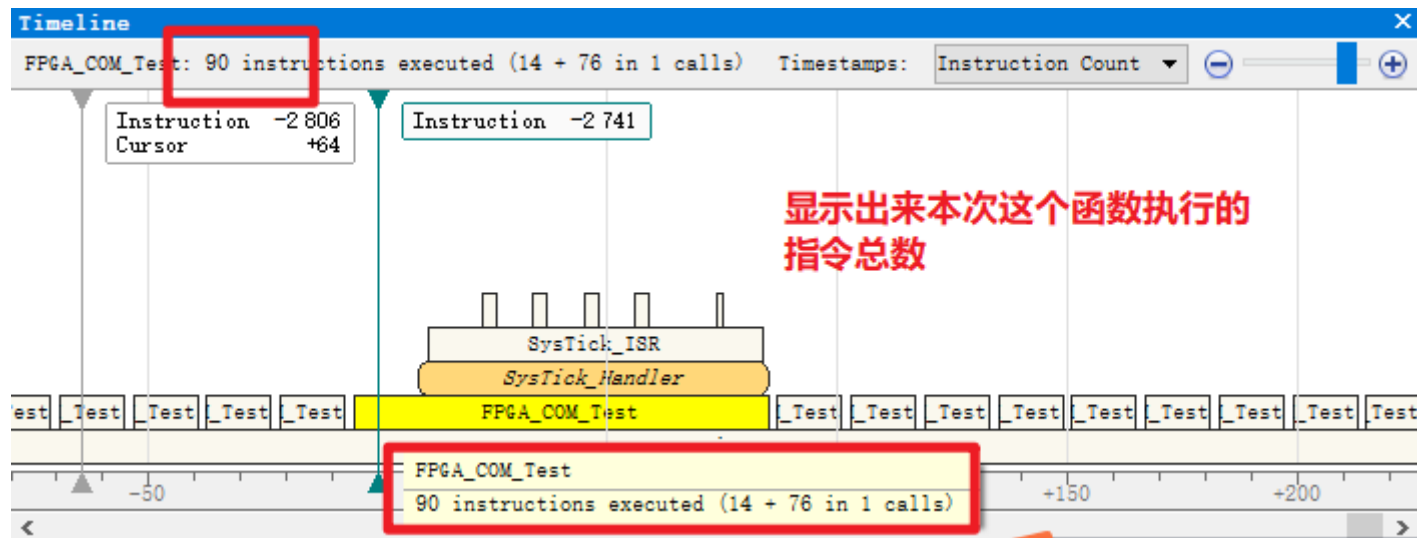
I At high levels of zoom, vertical line indicators are displayed inside of all call frames. Each neighboring pair of these instruction ticks bound the start and end of a single instruction execution.

In the example illustration to the right, the first instruction required 3 cycles to execute, while the following instructions were all executed in a single cycle.



以下是裸跑的情况





猜测STM32的指令都是一个指令周期相同，所以不存在不等分的情况

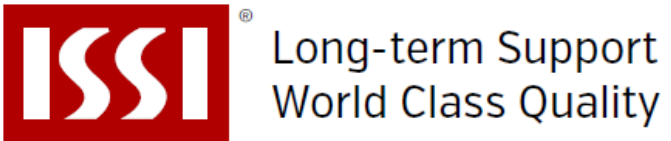
在时间线中选中任何一个函数的图框，都会跳转到该函数在文件窗口中的语句位置、反汇编窗口的指令位置、指令追踪窗口的计数位置，动图如下：

式)

遗憾的是，指令执行的上限，虽然在上图的最大指令计数上限，可以更改到比64M更多的数据，比如2.2G。但是就我实验发现，不论怎么设置，都无法突破64M左右的上限。因为购买这个J-Trace，看广告（产品介绍写的是无限跟踪），所以觉得不应该有这个问题。

当时一度以为64M这个坎是J-Trace的硬件性能决定的，因为当时拆解J-Trace时候发现J-Trace内部的SDRAM是1Gb（型号IS43LD32320），这样如果J-Trace实现的跟踪机制是先将ETM的情况记录在SDRAM中，当手动Ozone点击停止（或断点）后再将执行情况上报给Ozone的话，刨除掉ETM编码指令所浪费的字节，硬件上必然是有瓶颈的。这肯定是谈不上无限追踪的。

IS43/46LD16640C
IS43/46LD32320C



1Gb (x16, x32) Mobile LPDDR2 S4 SDRAM

AUGUST 2017

当时内心深处有一种被深深欺骗的感觉，但是又觉得Segger不至于这样吧。所以又做了个实验，USB不太会抓包。但是J-Trace是有网口的，可以通过Wireshark来抓包试试。

于是我根据这个想法做了个实验：

1.1、J-Trace和PC连接上之后的抓包结果，此时通讯数据速率不大

Capturing from Realtek Ethernet Controller (host 192.168.1.104) - Wireshark

No.	Time	Source	Destination	Protocol	Info
1	0.000000	30:9c:23:cf:6f:b7	SeggerMi_0e:01:75	ARP	Who has 192.168.1.104? Tell 192.168.1.66
2	0.000381	SeggerMi_0e:01:75	30:9c:23:cf:6f:b7	ARP	192.168.1.104 is at 00:22:c7:0e:01:75
3	0.068180	192.168.1.66	192.168.1.104	UDP	Source port: 53315 Destination port: 19020
4	0.068574	192.168.1.104	192.168.1.66	UDP	Source port: 19020 Destination port: 53315
5	0.336345	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=1 Ack=1 Win=62980 Len=6
6	0.336738	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=1 Ack=7 Win=4380 Len=128
7	0.376363	192.168.1.66	192.168.1.104	TCP	60298 > j-link [ACK] Seq=7 Ack=129 Win=62852 Len=0
8	1.069115	192.168.1.66	192.168.1.104	UDP	Source port: 55407 Destination port: 19020
9	1.069554	192.168.1.104	192.168.1.66	UDP	Source port: 19020 Destination port: 55407
10	1.087831	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=7 Ack=129 Win=62852 Len=4

1.2、点击绿色运行按钮，程序开始运行，此时通讯数据速率变大

[illegible]

2、程序开始运行后，开始大量出现UDP的通讯，这些udp的数据量还比较大1k级别的，应该是打包处理好的trace数据

Source IP	Destination IP	Source Port	Destination Port	Protocol
136 1.128337	192.168.1.104	192.168.1.66	UDP	Source port: 19030 Destination port: 64313
136 1.131105	192.168.1.104	192.168.1.66	UDP	Source port: 19030 Destination port: 64313
137 1.131318	192.168.1.104	192.168.1.66	UDP	Source port: 19030 Destination port: 64313

Frame 79: 1498 bytes on wire (11984 bits), 1498 bytes captured (11984 bits)
 Ethernet II, Src: SeggerMi_0e:01:75 (00:22:c7:0e:01:75), Dst: 30:9c:23:cf:6f:b7 (30:9c:23:cf:6f:b7)
 Destination: 30:9c:23:cf:6f:b7 (30:9c:23:cf:6f:b7)

1.3、等带一段时间后，点击暂停，Timeline窗口出现Trace的内容，拖动Timeline的窗口，追踪到的指令总数是67M左右，此时通讯数据速率也不大

Ozone - The J-Link Debugger V2.70e - C:/Users/Ning/Desktop/SPI407--20180615/Project/MDK-ARM(uV5)/Flash/Obj/output.axf

File View Find Debug Tools Window Help

Instruction Trace

PC	Op	Op	Op	Op	Op
08000426	BCC	0x08000416			__scatt
08000428	BL	__main_after_scatterload	0x08000190		
__main_after_scatterload					
08000190	LDR	RO, =0x08001C51	[0x08000194]		_main_init
08000192	BX	RO			
main					
08001C50	BL	bsp_Init	0x08001B14		

Code Profile

Function	Source Coverage	Inst. Coverage
* __lprintf\$3	N/A	0.0% (0/9)
* __aeabi_d2ulz	N/A	0.0% (0/16)
* __aeabi_dmul	N/A	0.0% (0/69)
* __aeabi_llsl	N/A	0.0% (0/12)

Functions

Name	Address	Size	#Insts	Source
* __lprintf\$3	0800 18C4	32	9	handlers.s
* __aeabi_d2ulz	0800 02BC	48	16	handlers.s
* __aeabi_dmul	0800 01C0	228	69	handlers.s
* __aeabi_llsl	0800 0318	30	12	handlers.s
* __aeabi_llsl	0800 0336	32	13	handlers.s

Console

```

Elf.GetBaseAddr(): // returns 0x80000000
Target.ReadU32 (0x08000000): // returns 0x200061
Target.SetReg ("SP", 0x200061D0):
Elf.GetEntryPointPC(): // returns 0x8000188
Target.SetReg ("PC", 0x8000188):
J-Link: Flash download: Bank 0 @ 0x80000000: Ski
Elf.GetBaseAddr(): // returns 0x80000000
Target.ReadU32 (0x08000000): // returns 0x200061
Target.SetReg ("SP", 0x200061D0):
Elf.GetEntryPointPC(): // returns 0x8000188
Target.SetReg ("PC", 0x8000188):
Executed J-Link command "SelectTraceSource=1"
Using dedicated IP streaming channel for max. tr
  
```

Timeline

File Scope

main

```

27 *****
28 * 函数名: main
29 * 功能说明: c程序入口
30 * 形参: 无
31 * 返回值: 错误代码(无需处理)
32 *****
33 /*
34 int main(void)
35 {
36 /*
37 ST固件库中的启动文件已经执行了 SystemInit() 函数, 该函数在 sys
38 配置CPU系统的时钟, 内部Flash访问时序, 配置FSMC用于外部SRAM
39 */
40
41 bsp_Init(); /* 硬件初始化 */
42 PrintfLogo(); /* 打印信息到串口1 */
43 while(1)
44 {
45     0800 1C58 B 0x08001C5E
46     0800 1C5E B 0x08001C5A
47     FPGA_COM_Test();
48 }
49
  
```

Disassembly

main

```

i.main
bsp_Init()
PrintfLogo
08001C54
while(1)
08001C58
FPGA_COM_T
08001C5A
while(1)
08001C5E
Region$$Ta
Data
08001C60
08001C64
08001C68
08001C6C
08001C70
08001C74
08001C78
08001C7C
Region$$Ta
08001C80
08001C84
08001C88
08001C8C
08001C90
08001C94
08001C98
  
```

PC

mit

rt2

oad

44164 13.474183	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44165 13.474187	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44166 13.474189	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	3、在最后暂停，等待Timeline
44167 13.474191	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	重建完所有分析后，立刻停止运行
44168 13.474193	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44169 13.474194	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	发现在最后这个过程中，抓包发现
44170 13.474196	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	数据在这个时间内，并没有大量
44171 13.474198	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44172 13.476249	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=1842 Ack=3229 Win=64220 Len=4		
44173 13.476822	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3229 Ack=1846 Win=4380 Len=4		
44174 13.476824	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44175 13.476893	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=1846 Ack=3233 Win=64216 Len=28		
44176 13.476920	192.168.1.66	192.168.1.104	UDP	Source port: 64313	Destination port: 19030	发送数据出来。
44177 13.477075	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44178 13.477077	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44179 13.477078	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44180 13.477079	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44181 13.477081	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44182 13.477084	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44183 13.477086	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44184 13.477088	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44185 13.479960	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3233 Ack=1874 Win=4380 Len=80		
44186 13.479964	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44187 13.480067	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=1874 Ack=3313 Win=64136 Len=12		
44188 13.480586	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3313 Ack=1886 Win=4380 Len=8		
44189 13.480614	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=1886 Ack=3321 Win=64128 Len=12		
44190 13.481118	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3321 Ack=1898 Win=4380 Len=8		
44191 13.481167	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=1898 Ack=3329 Win=64120 Len=73		
44192 13.482373	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3329 Ack=1971 Win=4380 Len=20		
44193 13.482424	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=1971 Ack=3349 Win=64100 Len=33		
44194 13.483114	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3349 Ack=2004 Win=4380 Len=12		
44195 13.483116	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44196 13.483162	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=2004 Ack=3361 Win=64088 Len=129		
44197 13.484853	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3361 Ack=2133 Win=4380 Len=36		
44198 13.484932	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=2133 Ack=3397 Win=64052 Len=3		
44199 13.485906	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44200 13.486118	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44201 13.486120	192.168.1.104	192.168.1.66	UDP	Source port: 19030	Destination port: 64313	
44202 13.486166	192.168.1.66	192.168.1.104	UDP	Source port: 64313	Destination port: 19030	
44203 13.486186	192.168.1.66	192.168.1.104	UDP	Source port: 64313	Destination port: 19030	
44204 13.486197	192.168.1.66	192.168.1.104	UDP	Source port: 64313	Destination port: 19030	
44205 13.486208	192.168.1.66	192.168.1.104	UDP	Source port: 64313	Destination port: 19030	
44206 13.486218	192.168.1.66	192.168.1.104	UDP	Source port: 64313	Destination port: 19030	
44207 13.486912	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3397 Ack=2136 Win=4380 Len=4		
44208 13.500863	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=2136 Ack=3401 Win=64048 Len=6		
44209 13.501375	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3401 Ack=2142 Win=4380 Len=128		
44210 13.503498	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=2142 Ack=3529 Win=63920 Len=12		
44211 13.504411	192.168.1.104	192.168.1.66	TCP	j-link > 60298 [PSH, ACK] Seq=3529 Ack=2154 Win=4380 Len=68		
44212 13.544841	192.168.1.66	192.168.1.104	TCP	60298 > j-link [ACK] Seq=2154 Ack=3597 Win=63852 Len=0		
44213 14.082428	192.168.1.66	192.168.1.104	UDP	Source port: 58896	Destination port: 19020	
44214 14.082866	192.168.1.104	192.168.1.66	UDP	Source port: 19020	Destination port: 58896	
44215 14.516767	192.168.1.66	192.168.1.104	TCP	60298 > j-link [PSH, ACK] Seq=2154 Ack=3597 Win=63852 Len=6		

所以最后我猜测，J-Trace在程序运行时就已经将跟踪数据不断的发送出来到PC了；
而不是将这些数据滚存在J-Trace的SDRAM中等调试结束后，再通过网线（或USB线）将滚存的追踪数据从SDRAM读取出来再发送。

为了解答我的疑惑，我给Segger发了个邮件，同时也问他们网站上写的关于无限跟踪是指什么。很快，当天就回复了，内容如下：

(基本印证了我的猜测)

Thank you for your inquiry.

The unlimited streaming trace feature is already available. You should see execution counters going up as well as code profiling and code coverage data.

<https://www.segger.com/products/development-tools/ozone-j-link-debugger/technology/trace-features/>

This is updated live and memorized unlimited.

However backtrace (the raw instruction trace data) is currently limited to the mentioned 64 MB due to legacy reasons in software.

It is planned to lift this limit. But keep in mind that if you capture multiple GB/TB of raw trace data you will need a very fast PC to analyze all this data in some reasonable time frame.

Currently we have no fixed time schedule as our project books are currently full with paid projects but this features are on our roadmap for next year.

On user side this will come as a firmware update on J-Trace PRO models and software update in Ozone/J-Link software for free so all you have to do is updated to the latest software when it is available.

To stay up to date with software updates you can subscribe here:

<https://www.segger.com/notification/subscribe.php?prodid=7>

Best regards,

后来又仔细看了看Segger的官网对Ozone的描述中，又看到点东西。老版的J-Trace应该是内置了存储空间（应该是RAM吧），具备跟踪64M指令的能力。

不知道是不是Ozone当前还没有兼容新款的硬件。我手里面这款J-Trace硬件相对来说是最新版本（固件也已经升级到了最新的DLL了）。

Requirements

The trace features can be used when the target system supports CPU trace. There are two trace methods:

Trace Pins

A **J-Trace** is connected to the trace pins of the target device. All trace data generated by the Embedded Trace Macrocell (ETM) is sent on these pins and recorded by J-Trace. With a J-Trace PRO the trace data is immediately sent to the host PC, which enables unlimited recording of the target execution and provides full system insight. **With an older J-Trace the trace data is stored in a 64 MB internal buffer and the latest trace data, usually about the last 2 seconds of execution, is read by the PC when the target is halted.**

Trace Buffer

难道是迁就老款，所以没有升级新款？

If the target device includes an Embedded Trace Buffer (ETB) or similar, trace can be done with a **J-Link**. The trace data is then written by the target into the internal trace buffer of usually about a few kilobytes and read by J-Link when the target is halted. With a trace buffer the last few thousand executed instructions can be available, which does not provide full insight, but can be useful to recall the execution flow before a fault or issue occurs.

跟踪上限这点I-Jet里面也有介绍：

Trace features

	I-jet	I-jet Trace CM L	I-jet Trace CM XL	I-jet Trace Arm
ETM trace buffer		64 MB	256 MB	256 MB
Trace data collection		1,2, and 4-bit	1,2, and 4-bit	1,2,4,8 and 16-bit
Maximum ETM trace clock		150 MHz	150 MHz	350 MHz
Maximum ETM bandwidth		1.2 Gbps	1.2 Gbps	11.2 Gbps
Automatic clock/data skew adjustment		✓	✓	✓
Automatic clock/data threshold adjustment				✓
ETB/MTB Trace support	✓	✓	✓	✓

2、指令跟踪窗口：

当程序停下来的时候（手动暂停或进入断点），当前PC指针在最底部，之前执行过的所有指令都会在该窗口下显现出来（隶属的函数语句在反汇编指令的右侧）。

USART1_IRQHandler					3
63 363 873	08001480	PUSH	{R4, LR}	{	
63 363 872	08001482	LDR	R0, =g_tUart1 ; [0x0800148C] =0x20000114	UartIRQ(&g_tUart1);	
63 363 871	08001484	BL	UartIRQ ; 0x08001686		
UartIRQ					5
63 363 870	08001686	PUSH	{R4, LR}	{	
63 363 869	08001688	MOV	R4, R0		
63 363 868	0800168A	MOVW	R1, #0x525	if (USART_GetITStatus(_pUart->uart, USART_IT_RXNE) != RESET)	
63 363 867	0800168E	LDR	R0, [R4]		
63 363 866	08001689	BL	USART_GetITStatus ; 0x08001502		
USART_GetITStatus					27
63 363 865	08001502	PUSH	{R4-R6, LR}	{	
63 363 864	08001504	MOV	R2, R0		
63 363 863	08001506	MOVS	R4, #0	uint32_t bitpos = 0x00, itmask = 0x00, usartreg = 0x00;	
63 363 862	08001508	MOVS	R3, #0		
63 363 861	0800150A	MOVS	R5, #0		
63 363 860	0800150C	MOVW	R0, #0	itstatus = itstatus = RESET;	
63 363 859	0800150E	MOVW	R6, #0x96A	if (USART_IT == USART_IT_CTS)	
63 363 858	08001512	CMP	R1, R6		
63 363 857	08001514	BNE	0x08001518		
63 363 856	08001518	UBFX	R5, R1, #5, #3	usartreg = ((uint8_t)USART_IT) >> 0x05;	
63 363 855	0800151C	AND	R1, R1, #31	itmask = USART_IT & IT_MASK;	
63 363 854	08001520	MOVS	R1, #1	itmask = (uint32_t)0x01 << itmask;	
63 363 853	08001522	LSL	R6, R3		
63 363 852	08001526	CMP	R5, #1	if (usartreg == 0x01) /* The IT is in CR1 register */	
63 363 851	08001528	BNE	0x08001530		
63 363 850	0800152A	LDRH	R6, [R2, #12]	itmask &= USARTx->CR1;	
63 363 849	0800152C	ANDS	R3, R6		
63 363 848	0800152E	B	0x0800153E		
63 363 847	0800153E	ASRS	R4, R1, #8	bitpos = USART_IT >> 0x08;	
63 363 846	08001540	MOVS	R6, #1	bitpos = (uint32_t)0x01 << bitpos;	
63 363 845	08001542	LSL.W	R4, R6, R4		
63 363 844	08001546	LDRH	R6, [R2]	bitpos &= USARTx->SR;	
63 363 843	08001548	ANDS	R4, R6		
63 363 842	0800154A	CBZ	R3, 0x08001552	if ((itmask != (uint16_t)RESET) && (bitpos != (uint16_t)RESET))	
63 363 841	0800154C	CBZ	R4, 0x08001552		
63 363 840	08001552	MOVS	R0, #0	bitstatus = RESET;	
63 363 839	08001554	POP	{R4-R6, PC}	}	
UartIRQ					4
63 363 838	08001694	CBZ	R0, 0x080016D0	UartIRQ	
63 363 837	080016D0	MOVW	R1, #0x727	if (USART_GetITStatus(_pUart->uart, USART_IT_TXE) != RESET)	
63 363 836	080016D4	LDR	R0, [R4]		
63 363 835	080016D6	BL	USART_GetITStatus ; 0x08001502		

上限由之前设置过的指令跟踪上限决定。和时间线窗口相同，点击后也是有着相同的交互关系。该窗口也支持指令根据函数折叠，动图如下：

因为有了跟踪，所以比起Debug来说，Trace可以展现出来语句（指令）执行的次数。之前使用Jlink的话，只能看到通过打断点，或者在已有程序里面加一个（更改程序，多多少少会改变实际程序的运行）计数器、打印等，才能得到执行次数。举一个（网上抄的）不恰当的例子，Debug本身更像是个照相机，但是Trace相对来说是个录像机了，可以帮忙我们更高的定位、排查问题。

The screenshot displays the J-Trace tool interface with three main panels:

- Left Panel (C Code):** Shows the source code for `RTOSInit_STM32F4xx.c`. A red box highlights the `if (SEGGER_SYSVIEW_DWT_IS_ENABLED() == 0u) {` block. A red arrow points to the execution statistics for this block.
- Bottom-Left Panel (Execution Statistics):** A table showing execution data for the highlighted code block:

Line	Not-Executed	Executed	Load
3238	0 (0.0%)	3238 (100.0%)	0.0%
- Right Panel (Instruction Trace):** Shows the assembly instructions corresponding to the C code. A red box highlights the instructions for the `if` block. A red arrow points to the disassembly view below.
- Bottom-Right Panel (Disassembly):** Shows the assembly instructions for the `if` block, including `LDR R3, #0x00000000`, `AND R3, R3, #1`, and `BNE 0x08000B6C`.

Red annotations and arrows provide additional context:

- A red arrow points to the execution statistics table with the text: "该语句执行的次数，橙色代表该语句或函数内有未被执行过的语句（指令）".
- A red arrow points to the `if` block in the C code with the text: "反汇编和函数语句是一一对应的".
- A red arrow points to the `if` block in the C code with the text: "鼠标悬停在执行次数上会给出一些更多的信息".
- A red arrow points to the `if` block in the C code with the text: "未被执行过".

在这两个窗口下，还可以设置Trace Start 和 Trace End。这个功能被Ozone 称作 Selective Tracing。可以只跟踪被选中的代码（可以选多组），这样Excute Count、Timeline和Instruction Trace都会只记录选中的代码。选择设置后，这些起止位置会在跟踪（断点）窗口中显示出来，同时支持勾选是否部分跟踪。

4.4.7 Selective Tracing

Ozone can instruct the target to constrain trace data output to individual address ranges (see *Tracepoints* on page 172). When selective tracing is active, it acts as a hardware prefilter of code profile data.

Ozone - The J-Link Debugger V2.70b - C:/Users/Ning/Desktop/SPI407--20180615/debug.jdebug

File View Find Debug Tools Window Help

Break & Tracepoints

Type	Location	Extras
☒ Code	bsp_fpga_com.c:111	case IDLE: /* 空闲状态, 等待新的中断触发 */
☒ Code	main	bsp_Init(): /* 硬件初始化 */
☐ Trace Start	bsp_fpga_com.c:112:4	
☐ Trace Stop	bsp_fpga_com.c:118:3	
☒ Trace Start	PrintfLogo	
☒ Trace Stop	main.c:70:1	

设置后同步显示在这里, 支持勾选

Disassembly

Address	OpCode	OpName	Comment
08000E84	BX	LR	
08000E86	NOP		
08000E88	PUSH	{R4, LR}	
08000E8A	ADD	R0, PC, #0x38	
08000E8C	BL	_2printf	; 0x080018E0
08000E8E	ADD	R0, PC, #0x34	
08000E90	ADD	R0, PC, #0x34	
08000E92	BL	_2printf	; 0x080018E0
08000E94	ADD	R0, PC, #0x34	
08000E96	ADD	R1, PC, #0x70	
08000E98	ADD	R0, PC, #0x80	
08000E9A	BL	_2printf	; 0x080018E0
08000E9C	ADD	R1, PC, #0x94	
08000E9E	ADD	R0, PC, #0x94	

PrintfLogo
i.PrintfLogo
{
08000E88 PUSH {R4, LR}
08000E8A ADD R0, PC, #0x38
08000E8C BL _2printf ; 0x080018E0
08000E8E ADD R0, PC, #0x34
08000E90 ADD R0, PC, #0x34
08000E92 BL _2printf ; 0x080018E0
08000E94 ADD R0, PC, #0x34
08000E96 ADD R1, PC, #0x70
08000E98 ADD R0, PC, #0x80
08000E9A BL _2printf ; 0x080018E0
08000E9C ADD R1, PC, #0x94
08000E9E ADD R0, PC, #0x94
08000E9F ADD R0, PC, #0x94

—对应

main.c

```
while(1)
{
    FPGA_COM_Test();
    // 设置跟踪起始和结束
    // *****
    // 函数名: PrintfLogo
    // 功能说明: 打印例程名
    // 形参: 无
    // 返回值: 无
    // *****
    static void PrintfLogo()
    {
        0800 0E88 PUSH
        0 60 printf("\n\r");
        0 0800 0E8A AC
        0 0800 0E8C BI
        0 61 printf("*****\n\r");
        0 62 printf("** 例程名称\n\r");
        0 63 printf("** 例程版本\n\r");
        0 64 printf("** 发布日期\n\r");
        65 // 打印STM32F4XX库版本
        0 67 printf("** STM32F4XX库版本\n\r");
        0 69 printf("*****\n\r");
        71 }
        72 //***** 安富莱电子 www.armfly.com
        73
```

Set Breakpoint
Set Tracepoint (Start)
Set Tracepoint (Stop)
Show Disassembly Ctrl+D
Graph Ctrl+G
Go To Line... Ctrl+L
Find... Ctrl+F
Expand Line Ctrl++
Expand All Alt++
Collapse All Alt+-
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Line Numbers
Execution Counters Ctrl+E
Instruction Encodings

这个操作完成后, Ozone会只显示和记录选中的跟踪内容, 毕竟不是无限记录 (可以省着点用)。

Instruction Trace

Function	Address
FPGA_COM_Test	20
bsp_DelayUS	1239
FPGA_COM_Test	20
bsp_DelayUS	1239
FPGA_COM_Test	20
bsp_DelayUS	1239
FPGA_COM_Test	20
bsp_DelayUS	983
SysTick_Handler	2
SysTick_ISR	12
bsp_SoftTimerDec	3
SysTick_ISR	8
bsp_SoftTimerDec	3
SysTick_ISR	8
bsp_SoftTimerDec	3
SysTick_ISR	8
bsp_SoftTimerDec	3
SysTick_ISR	15
bsp_RunPerims	1
SysTick_ISR	9
SysTick_Handler	1
bsp_DelayUS	152
FPGA_COM_Test	20
bsp_DelayUS	1239
FPGA_COM_Test	20
bsp_DelayUS	842

Timeline

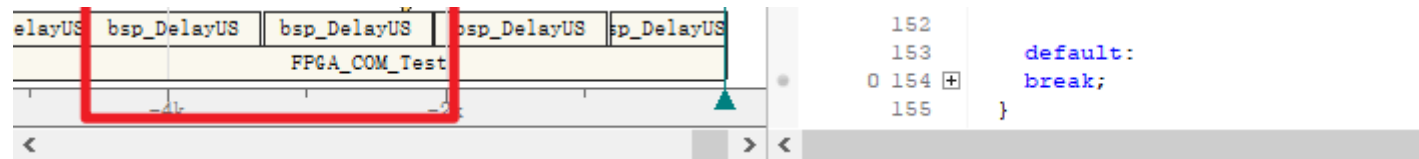
PC

也可以在程序窗口选择自己想追踪的代码 节省记录

File Scope

```

110 {
111     case IDLE:
112         __NOP();
113         crc_error_times++;
114         crc_ok_times++;
115         tmp_test++;
116         bsp_DelayUS(10);
117         crc_tmp++;
118         break;
119
120     case PULSE_GET:
121         // TEST_1();
122         bsp_DelayUS(5);
123         work_status = DATA_REV;
124
125     case DATA_COMPUTE:
126         crc_cal = 0;
127         for (tmp_i = 0; tmp_i < (S
128         {
129             crc_cal = crc_cal+spi_rx
130         }
131         if(crc_cal == spi_rx_buff[
132         { //和校验合法, 处理数据, 解析:
133             Compute_Freq(spi_rx_buff
134             //bsp_DelayUS(18);
135             crc_ok_times++;
136         }
137         else
138         { //和校验不合法, 直接结束
139             crc_error_times++;
140         }
141         work_status = IDLE;
142         break;
143
144     case DATA_COMPUTE:
145         crc_cal = 0;
146         for (tmp_i = 0; tmp_i < (S
147         {
148             crc_cal = crc_cal+spi_rx
149         }
150         if(crc_cal == spi_rx_buff[
151         { //和校验合法, 处理数据, 解析:
152             Compute_Freq(spi_rx_buff
153             //bsp_DelayUS(18);
154             crc_ok_times++;
155         }
156         else
157         { //和校验不合法, 直接结束
158             crc_error_times++;
159         }
160         work_status = IDLE;
161         break;
162
163     case DATA_COMPUTE:
164         crc_cal = 0;
165         for (tmp_i = 0; tmp_i < (S
166         {
167             crc_cal = crc_cal+spi_rx
168         }
169         if(crc_cal == spi_rx_buff[
170         { //和校验合法, 处理数据, 解析:
171             Compute_Freq(spi_rx_buff
172             //bsp_DelayUS(18);
173             crc_ok_times++;
174         }
175         else
176         { //和校验不合法, 直接结束
177             crc_error_times++;
178         }
179         work_status = IDLE;
180         break;
181
182     case DATA_COMPUTE:
183         crc_cal = 0;
184         for (tmp_i = 0; tmp_i < (S
185         {
186             crc_cal = crc_cal+spi_rx
187         }
188         if(crc_cal == spi_rx_buff[
189         { //和校验合法, 处理数据, 解析:
190             Compute_Freq(spi_rx_buff
191             //bsp_DelayUS(18);
192             crc_ok_times++;
193         }
194         else
195         { //和校验不合法, 直接结束
196             crc_error_times++;
197         }
198         work_status = IDLE;
199         break;
200
201     case DATA_COMPUTE:
202         crc_cal = 0;
203         for (tmp_i = 0; tmp_i < (S
204         {
205             crc_cal = crc_cal+spi_rx
206         }
207         if(crc_cal == spi_rx_buff[
208         { //和校验合法, 处理数据, 解析:
209             Compute_Freq(spi_rx_buff
210             //bsp_DelayUS(18);
211             crc_ok_times++;
212         }
213         else
214         { //和校验不合法, 直接结束
215             crc_error_times++;
216         }
217         work_status = IDLE;
218         break;
219
220     case DATA_COMPUTE:
221         crc_cal = 0;
222         for (tmp_i = 0; tmp_i < (S
223         {
224             crc_cal = crc_cal+spi_rx
225         }
226         if(crc_cal == spi_rx_buff[
227         { //和校验合法, 处理数据, 解析:
228             Compute_Freq(spi_rx_buff
229             //bsp_DelayUS(18);
230             crc_ok_times++;
231         }
232         else
233         { //和校验不合法, 直接结束
234             crc_error_times++;
235         }
236         work_status = IDLE;
237         break;
238
239     case DATA_COMPUTE:
240         crc_cal = 0;
241         for (tmp_i = 0; tmp_i < (S
242         {
243             crc_cal = crc_cal+spi_rx
244         }
245         if(crc_cal == spi_rx_buff[
246         { //和校验合法, 处理数据, 解析:
247             Compute_Freq(spi_rx_buff
248             //bsp_DelayUS(18);
249             crc_ok_times++;
250         }
251         else
252         { //和校验不合法, 直接结束
253             crc_error_times++;
254         }
255         work_status = IDLE;
256         break;
257
258     case DATA_COMPUTE:
259         crc_cal = 0;
260         for (tmp_i = 0; tmp_i < (S
261         {
262             crc_cal = crc_cal+spi_rx
263         }
264         if(crc_cal == spi_rx_buff[
265         { //和校验合法, 处理数据, 解析:
266             Compute_Freq(spi_rx_buff
267             //bsp_DelayUS(18);
268             crc_ok_times++;
269         }
270         else
271         { //和校验不合法, 直接结束
272             crc_error_times++;
273         }
274         work_status = IDLE;
275         break;
276
277     case DATA_COMPUTE:
278         crc_cal = 0;
279         for (tmp_i = 0; tmp_i < (S
280         {
281             crc_cal = crc_cal+spi_rx
282         }
283         if(crc_cal == spi_rx_buff[
284         { //和校验合法, 处理数据, 解析:
285             Compute_Freq(spi_rx_buff
286             //bsp_DelayUS(18);
287             crc_ok_times++;
288         }
289         else
290         { //和校验不合法, 直接结束
291             crc_error_times++;
292         }
293         work_status = IDLE;
294         break;
295
296     case DATA_COMPUTE:
297         crc_cal = 0;
298         for (tmp_i = 0; tmp_i < (S
299         {
300             crc_cal = crc_cal+spi_rx
301         }
302         if(crc_cal == spi_rx_buff[
303         { //和校验合法, 处理数据, 解析:
304             Compute_Freq(spi_rx_buff
305             //bsp_DelayUS(18);
306             crc_ok_times++;
307         }
308         else
309         { //和校验不合法, 直接结束
310             crc_error_times++;
311         }
312         work_status = IDLE;
313         break;
314
315     case DATA_COMPUTE:
316         crc_cal = 0;
317         for (tmp_i = 0; tmp_i < (S
318         {
319             crc_cal = crc_cal+spi_rx
320         }
321         if(crc_cal == spi_rx_buff[
322         { //和校验合法, 处理数据, 解析:
323             Compute_Freq(spi_rx_buff
324             //bsp_DelayUS(18);
325             crc_ok_times++;
326         }
327         else
328         { //和校验不合法, 直接结束
329             crc_error_times++;
330         }
331         work_status = IDLE;
332         break;
333
334     case DATA_COMPUTE:
335         crc_cal = 0;
336         for (tmp_i = 0; tmp_i < (S
337         {
338             crc_cal = crc_cal+spi_rx
339         }
340         if(crc_cal == spi_rx_buff[
341         { //和校验合法, 处理数据, 解析:
342             Compute_Freq(spi_rx_buff
343             //bsp_DelayUS(18);
344             crc_ok_times++;
345         }
346         else
347         { //和校验不合法, 直接结束
348             crc_error_times++;
349         }
350         work_status = IDLE;
351         break;
352
353     case DATA_COMPUTE:
354         crc_cal = 0;
355         for (tmp_i = 0; tmp_i < (S
356         {
357             crc_cal = crc_cal+spi_rx
358         }
359         if(crc_cal == spi_rx_buff[
360         { //和校验合法, 处理数据, 解析:
361             Compute_Freq(spi_rx_buff
362             //bsp_DelayUS(18);
363             crc_ok_times++;
364         }
365         else
366         { //和校验不合法, 直接结束
367             crc_error_times++;
368         }
369         work_status = IDLE;
370         break;
371
372     case DATA_COMPUTE:
373         crc_cal = 0;
374         for (tmp_i = 0; tmp_i < (S
375         {
376             crc_cal = crc_cal+spi_rx
377         }
378         if(crc_cal == spi_rx_buff[
379         { //和校验合法, 处理数据, 解析:
380             Compute_Freq(spi_rx_buff
381             //bsp_DelayUS(18);
382             crc_ok_times++;
383         }
384         else
385         { //和校验不合法, 直接结束
386             crc_error_times++;
387         }
388         work_status = IDLE;
389         break;
390
391     case DATA_COMPUTE:
392         crc_cal = 0;
393         for (tmp_i = 0; tmp_i < (S
394         {
395             crc_cal = crc_cal+spi_rx
396         }
397         if(crc_cal == spi_rx_buff[
398         { //和校验合法, 处理数据, 解析:
399             Compute_Freq(spi_rx_buff
400             //bsp_DelayUS(18);
401             crc_ok_times++;
402         }
403         else
404         { //和校验不合法, 直接结束
405             crc_error_times++;
406         }
407         work_status = IDLE;
408         break;
409
410     case DATA_COMPUTE:
411         crc_cal = 0;
412         for (tmp_i = 0; tmp_i < (S
413         {
414             crc_cal = crc_cal+spi_rx
415         }
416         if(crc_cal == spi_rx_buff[
417         { //和校验合法, 处理数据, 解析:
418             Compute_Freq(spi_rx_buff
419             //bsp_DelayUS(18);
420             crc_ok_times++;
421         }
422         else
423         { //和校验不合法, 直接结束
424             crc_error_times++;
425         }
426         work_status = IDLE;
427         break;
428
429     case DATA_COMPUTE:
430         crc_cal = 0;
431         for (tmp_i = 0; tmp_i < (S
432         {
433             crc_cal = crc_cal+spi_rx
434         }
435         if(crc_cal == spi_rx_buff[
436         { //和校验合法, 处理数据, 解析:
437             Compute_Freq(spi_rx_buff
438             //bsp_DelayUS(18);
439             crc_ok_times++;
440         }
441         else
442         { //和校验不合法, 直接结束
443             crc_error_times++;
444         }
445         work_status = IDLE;
446         break;
447
448     case DATA_COMPUTE:
449         crc_cal = 0;
450         for (tmp_i = 0; tmp_i < (S
451         {
452             crc_cal = crc_cal+spi_rx
453         }
454         if(crc_cal == spi_rx_buff[
455         { //和校验合法, 处理数据, 解析:
456             Compute_Freq(spi_rx_buff
457             //bsp_DelayUS(18);
458             crc_ok_times++;
459         }
460         else
461         { //和校验不合法, 直接结束
462             crc_error_times++;
463         }
464         work_status = IDLE;
465         break;
466
467     case DATA_COMPUTE:
468         crc_cal = 0;
469         for (tmp_i = 0; tmp_i < (S
470         {
471             crc_cal = crc_cal+spi_rx
472         }
473         if(crc_cal == spi_rx_buff[
474         { //和校验合法, 处理数据, 解析:
475             Compute_Freq(spi_rx_buff
476             //bsp_DelayUS(18);
477             crc_ok_times++;
478         }
479         else
480         { //和校验不合法, 直接结束
481             crc_error_times++;
482         }
483         work_status = IDLE;
484         break;
485
486     case DATA_COMPUTE:
487         crc_cal = 0;
488         for (tmp_i = 0; tmp_i < (S
489         {
490             crc_cal = crc_cal+spi_rx
491         }
492         if(crc_cal == spi_rx_buff[
493         { //和校验合法, 处理数据, 解析:
494             Compute_Freq(spi_rx_buff
495             //bsp_DelayUS(18);
496             crc_ok_times++;
497         }
498         else
499         { //和校验不合法, 直接结束
500             crc_error_times++;
501         }
502         work_status = IDLE;
503         break;
504
505     case DATA_COMPUTE:
506         crc_cal = 0;
507         for (tmp_i = 0; tmp_i < (S
508         {
509             crc_cal = crc_cal+spi_rx
510         }
511         if(crc_cal == spi_rx_buff[
512         { //和校验合法, 处理数据, 解析:
513             Compute_Freq(spi_rx_buff
514             //bsp_DelayUS(18);
515             crc_ok_times++;
516         }
517         else
518         { //和校验不合法, 直接结束
519             crc_error_times++;
520         }
521         work_status = IDLE;
522         break;
523
524     case DATA_COMPUTE:
525         crc_cal = 0;
526         for (tmp_i = 0; tmp_i < (S
527         {
528             crc_cal = crc_cal+spi_rx
529         }
530         if(crc_cal == spi_rx_buff[
531         { //和校验合法, 处理数据, 解析:
532             Compute_Freq(spi_rx_buff
533             //bsp_DelayUS(18);
534             crc_ok_times++;
535         }
536         else
537         { //和校验不合法, 直接结束
538             crc_error_times++;
539         }
540         work_status = IDLE;
541         break;
542
543     case DATA_COMPUTE:
544         crc_cal = 0;
545         for (tmp_i = 0; tmp_i < (S
546         {
547             crc_cal = crc_cal+spi_rx
548         }
549         if(crc_cal == spi_rx_buff[
550         { //和校验合法, 处理数据, 解析:
551             Compute_Freq(spi_rx_buff
552             //bsp_DelayUS(18);
553             crc_ok_times++;
554         }
555         else
556         { //和校验不合法, 直接结束
557             crc_error_times++;
558         }
559         work_status = IDLE;
560         break;
561
562     case DATA_COMPUTE:
563         crc_cal = 0;
564         for (tmp_i = 0; tmp_i < (S
565         {
566             crc_cal = crc_cal+spi_rx
567         }
568         if(crc_cal == spi_rx_buff[
569         { //和校验合法, 处理数据, 解析:
570             Compute_Freq(spi_rx_buff
571             //bsp_DelayUS(18);
572             crc_ok_times++;
573         }
574         else
575         { //和校验不合法, 直接结束
576             crc_error_times++;
577         }
578         work_status = IDLE;
579         break;
580
581     case DATA_COMPUTE:
582         crc_cal = 0;
583         for (tmp_i = 0; tmp_i < (S
584         {
585             crc_cal = crc_cal+spi_rx
586         }
587         if(crc_cal == spi_rx_buff[
588         { //和校验合法, 处理数据, 解析:
589             Compute_Freq(spi_rx_buff
590             //bsp_DelayUS(18);
591             crc_ok_times++;
592         }
593         else
594         { //和校验不合法, 直接结束
595             crc_error_times++;
596         }
597         work_status = IDLE;
598         break;
599
600     case DATA_COMPUTE:
601         crc_cal = 0;
602         for (tmp_i = 0; tmp_i < (S
603         {
604             crc_cal = crc_cal+spi_rx
605         }
606         if(crc_cal == spi_rx_buff[
607         { //和校验合法, 处理数据, 解析:
608             Compute_Freq(spi_rx_buff
609             //bsp_DelayUS(18);
610             crc_ok_times++;
611         }
612         else
613         { //和校验不合法, 直接结束
614             crc_error_times++;
615         }
616         work_status = IDLE;
617         break;
618
619     case DATA_COMPUTE:
620         crc_cal = 0;
621         for (tmp_i = 0; tmp_i < (S
622         {
623             crc_cal = crc_cal+spi_rx
624         }
625         if(crc_cal == spi_rx_buff[
626         { //和校验合法, 处理数据, 解析:
627             Compute_Freq(spi_rx_buff
628             //bsp_DelayUS(18);
629             crc_ok_times++;
630         }
631         else
632         { //和校验不合法, 直接结束
633             crc_error_times++;
634         }
635         work_status = IDLE;
636         break;
637
638     case DATA_COMPUTE:
639         crc_cal = 0;
640         for (tmp_i = 0; tmp_i < (S
641         {
642             crc_cal = crc_cal+spi_rx
643         }
644         if(crc_cal == spi_rx_buff[
645         { //和校验合法, 处理数据, 解析:
646             Compute_Freq(spi_rx_buff
647             //bsp_DelayUS(18);
648             crc_ok_times++;
649         }
650         else
651         { //和校验不合法, 直接结束
652             crc_error_times++;
653         }
654         work_status = IDLE;
655         break;
656
657     case DATA_COMPUTE:
658         crc_cal = 0;
659         for (tmp_i = 0; tmp_i < (S
660         {
661             crc_cal = crc_cal+spi_rx
662         }
663         if(crc_cal == spi_rx_buff[
664         { //和校验合法, 处理数据, 解析:
665             Compute_Freq(spi_rx_buff
666             //bsp_DelayUS(18);
667             crc_ok_times++;
668         }
669         else
670         { //和校验不合法, 直接结束
671             crc_error_times++;
672         }
673         work_status = IDLE;
674         break;
675
676     case DATA_COMPUTE:
677         crc_cal = 0;
678         for (tmp_i = 0; tmp_i < (S
679         {
680             crc_cal = crc_cal+spi_rx
681         }
682         if(crc_cal == spi_rx_buff[
683         { //和校验合法, 处理数据, 解析:
684             Compute_Freq(spi_rx_buff
685             //bsp_DelayUS(18);
686             crc_ok_times++;
687         }
688         else
689         { //和校验不合法, 直接结束
690             crc_error_times++;
691         }
692         work_status = IDLE;
693         break;
694
695     case DATA_COMPUTE:
696         crc_cal = 0;
697         for (tmp_i = 0; tmp_i < (S
698         {
699             crc_cal = crc_cal+spi_rx
700         }
701         if(crc_cal == spi_rx_buff[
702         { //和校验合法, 处理数据, 解析:
703             Compute_Freq(spi_rx_buff
704             //bsp_DelayUS(18);
705             crc_ok_times++;
706         }
707         else
708         { //和校验不合法, 直接结束
709             crc_error_times++;
710         }
711         work_status = IDLE;
712         break;
713
714     case DATA_COMPUTE:
715         crc_cal = 0;
716         for (tmp_i = 0; tmp_i < (S
717         {
718             crc_cal = crc_cal+spi_rx
719         }
720         if(crc_cal == spi_rx_buff[
721         { //和校验合法, 处理数据, 解析:
722             Compute_Freq(spi_rx_buff
723             //bsp_DelayUS(18);
724             crc_ok_times++;
725         }
726         else
727         { //和校验不合法, 直接结束
728             crc_error_times++;
729         }
730         work_status = IDLE;
731         break;
732
733     case DATA_COMPUTE:
734         crc_cal = 0;
735         for (tmp_i = 0; tmp_i < (S
736         {
737             crc_cal = crc_cal+spi_rx
738         }
739         if(crc_cal == spi_rx_buff[
740         { //和校验合法, 处理数据, 解析:
741             Compute_Freq(spi_rx_buff
742             //bsp_DelayUS(18);
743             crc_ok_times++;
744         }
745         else
746         { //和校验不合法, 直接结束
747             crc_error_times++;
748         }
749         work_status = IDLE;
750         break;
751
752     case DATA_COMPUTE:
753         crc_cal = 0;
754         for (tmp_i = 0; tmp_i < (S
755         {
756             crc_cal = crc_cal+spi_rx
757         }
758         if(crc_cal == spi_rx_buff[
759         { //和校验合法, 处理数据, 解析:
760             Compute_Freq(spi_rx_buff
761             //bsp_DelayUS(18);
762             crc_ok_times++;
763         }
764         else
765         { //和校验不合法, 直接结束
766             crc_error_times++;
767         }
768         work_status = IDLE;
769         break;
770
771     case DATA_COMPUTE:
772         crc_cal = 0;
773         for (tmp_i = 0; tmp_i < (S
774         {
775             crc_cal = crc_cal+spi_rx
776         }
777         if(crc_cal == spi_rx_buff[
778         { //和校验合法, 处理数据, 解析:
779             Compute_Freq(spi_rx_buff
780             //bsp_DelayUS(18);
781             crc_ok_times++;
782         }
783         else
784         { //和校验不合法, 直接结束
785             crc_error_times++;
786         }
787         work_status = IDLE;
788         break;
789
790     case DATA_COMPUTE:
791         crc_cal = 0;
792         for (tmp_i = 0; tmp_i < (S
793         {
794             crc_cal = crc_cal+spi_rx
795         }
796         if(crc_cal == spi_rx_buff[
797         { //和校验合法, 处理数据, 解析:
798             Compute_Freq(spi_rx_buff
799             //bsp_DelayUS(18);
800             crc_ok_times++;
801         }
802         else
803         { //和校验不合法, 直接结束
804             crc_error_times++;
805         }
806         work_status = IDLE;
807         break;
808
809     case DATA_COMPUTE:
810         crc_cal = 0;
811         for (tmp_i = 0; tmp_i < (S
812         {
813             crc_cal = crc_cal+spi_rx
814         }
815         if(crc_cal == spi_rx_buff[
816         { //和校验合法, 处理数据, 解析:
817             Compute_Freq(spi_rx_buff
818             //bsp_DelayUS(18);
819             crc_ok_times++;
820         }
821         else
822         { //和校验不合法, 直接结束
823             crc_error_times++;
824         }
825         work_status = IDLE;
826         break;
827
828     case DATA_COMPUTE:
829         crc_cal = 0;
830         for (tmp_i = 0; tmp_i < (S
831         {
832             crc_cal = crc_cal+spi_rx
833         }
834         if(crc_cal == spi_rx_buff[
835         { //和校验合法, 处理数据, 解析:
836             Compute_Freq(spi_rx_buff
837             //bsp_DelayUS(18);
838             crc_ok_times++;
839         }
840         else
841         { //和校验不合法, 直接结束
842             crc_error_times++;
843         }
844         work_status = IDLE;
845         break;
846
847     case DATA_COMPUTE:
848         crc_cal = 0;
849         for (tmp_i = 0; tmp_i < (S
850         {
851             crc_cal = crc_cal+spi_rx
852         }
853         if(crc_cal == spi_rx_buff[
854         { //和校验合法, 处理数据, 解析:
855             Compute_Freq(spi_rx_buff
856             //bsp_DelayUS(18);
857             crc_ok_times++;
858         }
859         else
860         { //和校验不合法, 直接结束
861             crc_error_times++;
862         }
863         work_status = IDLE;
864         break;
865
866     case DATA_COMPUTE:
867         crc_cal = 0;
868         for (tmp_i = 0; tmp_i < (S
869         {
870             crc_cal = crc_cal+spi_rx
871         }
872         if(crc_cal == spi_rx_buff[
873         { //和校验合法, 处理数据, 解析:
874             Compute_Freq(spi_rx_buff
875             //bsp_DelayUS(18);
876             crc_ok_times++;
877         }
878         else
879         { //和校验不合法, 直接结束
880             crc_error_times++;
881         }
882         work_status = IDLE;
883         break;
884
885     case DATA_COMPUTE:
886         crc_cal = 0;
887         for (tmp_i = 0; tmp_i < (S
888         {
889             crc_cal = crc_cal+spi_rx
890         }
891         if(crc_cal == spi_rx_buff[
892         { //和校验合法, 处理数据, 解析:
893             Compute_Freq(spi_rx_buff
894             //bsp_DelayUS(18);
895             crc_ok_times++;
896         }
897         else
898         { //和校验不合法, 直接结束
899             crc_error_times++;
900         }
901         work_status = IDLE;
902         break;
903
904     case DATA_COMPUTE:
905         crc_cal = 0;
906         for (tmp_i = 0; tmp_i < (S
907         {
908             crc_cal = crc_cal+spi_rx
909         }
910         if(crc_cal == spi_rx_buff[
911         { //和校验合法, 处理数据, 解析:
912             Compute_Freq(spi_rx_buff
913             //bsp_DelayUS(18);
914             crc_ok_times++;
915         }
916         else
917         { //和校验不合法, 直接结束
918             crc_error_times++;
919         }
920         work_status = IDLE;
921         break;
922
923     case DATA_COMPUTE:
924         crc_cal = 0;
925         for (tmp_i = 0; tmp_i < (S
926         {
927             crc_cal = crc_cal+spi_rx
928         }
929         if(crc_cal == spi_rx_buff[
930         { //和校验合法, 处理数据, 解析:
931             Compute_Freq(spi_rx_buff
932             //bsp_DelayUS(18);
933             crc_ok_times++;
934         }
935         else
936         { //和校验不合法, 直接结束
937             crc_error_times++;
938         }
939         work_status = IDLE;
940         break;
941
942     case DATA_COMPUTE:
943         crc_cal = 0;
944         for (tmp_i = 0; tmp_i < (S
945         {
946             crc_cal = crc_cal+spi_rx
947         }
948         if(crc_cal == spi_rx_buff[
949         { //和校验合法, 处理数据, 解析:
950             Compute_Freq(spi_rx_buff
951             //bsp_DelayUS(18);
952             crc_ok_times++;
953         }
954         else
955         { //和校验不合法, 直接结束
956             crc_error_times++;
957         }
958         work_status = IDLE;
959         break;
960
961     case DATA_COMPUTE:
962         crc_cal = 0;
963         for (tmp_i = 0; tmp_i < (S
964         {
965             crc_cal = crc_cal+spi_rx
966         }
967         if(crc_cal == spi_rx_buff[
968         { //和校验合法, 处理数据, 解析:
969             Compute_Freq(spi_rx_buff
970             //bsp_DelayUS(18);
971             crc_ok_times++;
972         }
973         else
974         { //和校验不合法, 直接结束
975             crc_error_times++;
976         }
977         work_status = IDLE;
978         break;
979
980     case DATA_COMPUTE:
981         crc_cal = 0;
982         for (tmp_i = 0; tmp_i < (S
983         {
984             crc_cal = crc_cal+spi_rx
985         }
986         if(crc_cal == spi_rx_buff[
987         { //和校验合法, 处理数据, 解析:
988             Compute_Freq(spi_rx_buff
989             //bsp_DelayUS(18);
990             crc_ok_times++;
991         }
992         else
993         { //和校验不合法, 直接结束
994             crc_error_times++;
995         }
996         work_status = IDLE;
997         break;
998
999     case DATA_COMPUTE:
1000         crc_cal = 0;
1001         for (tmp_i = 0; tmp_i < (S
1002         {
1003             crc_cal = crc_cal+spi_rx
1004         }
1005         if(crc_cal == spi_rx_buff[
1006         { //和校验合法, 处理数据, 解析:
1007             Compute_Freq(spi_rx_buff
1008             //bsp_DelayUS(18);
1009             crc_ok_times++;
1010         }
1011         else
1012         { //和校验不合法, 直接结束
1013             crc_error_times++;
1014         }
1015         work_status = IDLE;
1016         break;
1017
1018     case DATA_COMPUTE:
1019         crc_cal = 0;
1020         for (tmp_i = 0; tmp_i < (S
1021         {
1022             crc_cal = crc_cal+spi_rx
1023         }
1024         if(crc_cal == spi_rx_buff[
1025         { //和校验合法, 处理数据, 解析:
1026             Compute_Freq(spi_rx_buff
1027             //bsp_DelayUS(18);
1028             crc_ok_times++;
1029         }
1030         else
1031         { //和校验不合法, 直接结束
1032             crc_error_times++;
1033         }
1034         work_status = IDLE;
1035         break;
1036
1037     case DATA_COMPUTE:
1038         crc_cal = 0;
1039         for (tmp_i = 0; tmp_i < (S
1040         {
1041             crc_cal = crc_cal+spi_rx
1042         }
1043         if(crc_cal == spi_rx_buff[
1044         { //和校验合法, 处理数据, 解析:
1045             Compute_Freq(spi_rx_buff
1046             //bsp_DelayUS(18);
1047             crc_ok_times++;
1048         }
1049         else
1050         { //和校验不合法, 直接结束
1051             crc_error_times++;
1052         }
1053         work_status = IDLE;
1054         break;
1055
1056     case DATA_COMPUTE:
1057         crc_cal = 0;
1058         for (tmp_i = 0; tmp_i < (S
1059         {
1060             crc_cal = crc_cal+spi_rx
1061         }
1062         if(crc_cal == spi_rx_buff[
1063         { //和校验合法, 处理数据, 解析:
1064             Compute_Freq(spi_rx_buff
1065             //bsp_DelayUS(18);
1066             crc_ok_times++;
1067         }
1068         else
1069         { //和校验不合法, 直接结束
1070             crc_error_times++;
1071         }
1072         work_status = IDLE;
1073         break;
1074
1075     case DATA_COMPUTE:
1076         crc_cal = 0;
1077         for (tmp_i = 0; tmp_i < (S
1078         {
1079             crc_cal = crc_cal+spi_rx
1080         }
1081         if(crc_cal == spi_rx_buff[
1082         { //和校验合法, 处理数据, 解析:
1083             Compute_Freq(spi_rx_buff
1084             //bsp_DelayUS(18);
1085             crc_ok_times++;
1086         }
1087         else
1088         { //和校验不合法, 直接结束
1089             crc_error_times++;
1090         }
1091         work_status = IDLE;
1092         break;
1093
1094     case DATA_COMPUTE:
1095         crc_cal = 0;
1096         for (tmp_i = 0; tmp_i < (S
1097         {
1098             crc_cal = crc_cal+spi_rx
1099         }
1100         if(crc_cal == spi_rx_buff[
1101         { //和校验合法, 处理数据, 解析:
1102             Compute_Freq(spi_rx_buff
1103             //bsp_DelayUS(18);
1104             crc_ok_times++;
1105         }
1106         else
1107         { //和校验不合法, 直接结束
1108             crc_error_times++;
1109         }
1110         work_status = IDLE;
1111         break;
1112
1113     case DATA_COMPUTE:
1114         crc_cal = 0;
1115         for (tmp_i = 0; tmp_i < (S
1116         {
1117             crc_cal = crc_cal+spi_rx
1118         }
1119         if(crc_cal == spi_rx_buff[
1120         { //和校验合法, 处理数据, 解析:
1121             Compute_Freq(spi_rx_buff
1122             //bsp_DelayUS(18);
1123             crc_ok_times++;
1124         }
1125         else
1126         { //和校验不合法, 直接结束
1127             crc_error_times++;
1128         }
1129         work_status = IDLE;
1130         break;
1131
1132     case DATA_COMPUTE:
1133         crc_cal = 0;
1134         for (tmp_i = 0; tmp_i < (S
1135         {
1136             crc_cal = crc_cal+spi_rx
1137         }
1138         if(crc_cal == spi_rx_buff[
1139         { //和校验合法, 处理数据, 解析:
1140             Compute_Freq(spi_rx_buff
1141             //bsp_DelayUS(18);
1142             crc_ok_times++;
1143         }
1144         else
1145         { //和校验不合法, 直接结束
1146             crc_error_times++;
1147         }
1148         work_status = IDLE;
1149         break;
1150
1151     case DATA_COMPUTE:
1152         crc_cal = 0;
1153         for (tmp_i = 0; tmp_i < (S
1154         {
1155             crc_cal = crc_cal+spi_rx
1156         }
1157         if(crc_cal == spi_rx_buff[
1158         { //和校验合法, 处理数据, 解析:
1159             Compute_Freq(spi_rx_buff
1160             //bsp_DelayUS(18);
1161             crc_ok_times++;
1162         }
1163         else
1164         { //和校验不合法, 直接结束
1165             crc_error_times++;
1166         }
1167         work_status = IDLE;
1168         break;
1169
1170     case DATA_COMPUTE:
1171         crc_cal = 0;
1172         for (tmp_i = 0; tmp_i < (S
1173         {
1174             crc_cal = crc_cal+spi_rx
1175         }
1176         if(crc_cal == spi_rx_buff[
1177         { //和校验合法, 处理数据, 解析:
1178             Compute_Freq(spi_rx_buff
1179             //bsp_DelayUS(18);
1180             crc_ok_times++;
1181         }
1182         else
1183         { //和校验不合法, 直接结束
1184             crc_error_times++;
1185         }
1186         work_status = IDLE;
1187         break;
1188
1189     case DATA_COMPUTE:
1190         crc_cal = 0;
1191         for (tmp_i = 0; tmp_i < (S
1192         {
1193             crc_cal = crc_cal+spi_rx
1194         }
1195         if(crc_cal == spi_rx_buff[
1196         { //和校验合法, 处理数据, 解析:
1197             Compute_Freq(spi_rx_buff
1198             //bsp_DelayUS(18);
1199             crc_ok_times++;
1200         }
1201         else
1202         { //和校验不合法, 直接结束
12
```



5、代码执行状态窗口：

可以看到函数语句的执行覆盖率、反汇编后指令的执行覆盖率、以及Run Count和Fetch Count，还有就是负载率了。动图如下：

Code Profile							
Function	Source Coverage	Inst. Coverage		Run Count	Fetch Count	Load	
*	*	*		*	*	*	
☒ f WWDG_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f USART6_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f USART3_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f USART2_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f USART1_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f UsageFault_Handler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f UART5_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f UART4_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM8_UP_TIM13_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM8_TRG_COM_TIM14_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM8_CC_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM8_BRK_TIM12_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM7_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM6_DAC_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM5_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM4_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM3_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM2_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM1_UP_TIM10_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM1_TRG_COM_TIM11_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM1_CC_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TIM1_BRK_TIM9_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f TAMP_STAMP_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f SysTick_Handler	0.0% (0/8)	☒	0.0% (0/113)	0	0	☒	0.00%
☒ f SysTick_Handler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f SysTick_Config	0.0% (0/9)	☒	0.0% (0/26)	0	0	☒	0.00%
☒ f SystemInit	0.0% (0/11)	☒	0.0% (0/37)	0	0	☒	0.00%
☒ f SystemCoreClockUpdate	0.0% (0/21)	☒	0.0% (0/97)	0	0	☒	0.00%
☒ f SVC_Handler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f strcpy	N/A	☒	0.0% (0/22)	0	0	☒	0.00%
☒ f start	0.0% (0/4)	☒	0.0% (0/4)	0	0	☒	0.00%
☒ f SPI3_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f SPI2_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f SPI1_IRQHandler	0.0% (0/1)	☒	0.0% (0/1)	0	0	☒	0.00%
☒ f SetSysClock	0.0% (0/23)	☒	0.0% (0/96)	0	0	☒	0.00%
☒ f SEGGER_SYSVIEW_Stop	0.0% (0/7)	☒	0.0% (0/25)	0	0	☒	0.00%
☒ f SEGGER_SYSVIEW_Start	0.0% (0/21)	☒	0.0% (0/161)	0	0	☒	0.00%
☒ f SEGGER_SYSVIEW_ShieldId	0.0% (0/2)	☒	0.0% (0/10)	0	0	☒	0.00%

其中，如果不想看某些空闲函数的CPU负载率的话，可以去掉这个函数的负载率计算。这个在使用OS的时候可以去掉Idle任务。点

开函数的折叠，也可以看到具体的指令执行情况。根据Segger回复我的邮件，这个过程应该是可以无限追踪的

感觉Fetch Count是函数中的C语句每条汇编后的指令执行次数之和，所以次数会远远大于函数本身的Run Count。不知道这个理解对不对。

Code Profile						
Function	Source Coverage	Inst. Coverage	Run Count	Fetch Count	Load	
*	*	*	*	*	*	
f bsp_DelayUS	93.8% (15/16)	✓ 97.0% (32/33)	712 545	104 685 688	✓ 84.41%	
f FPM32M_Test	28.6% (8/28)	✓ 23.4% (25/107)	712 545	17 813 625	✓ 14.36%	
C 114: tmp_test++;	100.0% (1/1)	✓ 100.0% (5/5)	712 545	3 562 725	✓ 2.87%	
0800 07A4 STRB R0, [R1]	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 07A2 LDR R1, =tmp_test ;	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 07A0 ADDS R0, R0, #1	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 079E LDRB R0, [R0]	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 079C LDR R0, =tmp_test ;	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
C 113: crc_ok_times++;	100.0% (1/1)	✓ 100.0% (5/5)	712 545	3 562 725	✓ 2.87%	
0800 079A STRH R0, [R1]	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 0798 LDR R1, =crc_ok_time	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 0796 ADDS R0, R0, #1	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 0794 LDRH R0, [R0]	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 0792 LDR R0, =crc_ok_time	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
C 112: crc_error_times++;	100.0% (1/1)	✓ 100.0% (5/5)	712 545	3 562 725	✓ 2.87%	
0800 0790 STRB R0, [R1]	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 078E LDR R1, =crc_error_t	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 078C ADDS R0, R0, #1	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 078A LDR R0, [R0]	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 0788 LDR R0, =crc_error_t	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
C 108: switch (work_status)	0.0% (0/1)	✓ 30.0% (3/10)	712 545	2 137 635	✓ 1.72%	
0800 0776 CBZ R0, 0x00000786	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 0774 LDRB R0, [R0]	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 0772 LDR R0, =work_status	N/A	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
0800 0784 B 0x080007E0	N/A	✓ 0.0% (0/1)	0	0	✓ 0.00%	
0800 0782 BNE 0x0800084C	N/A	✓ 0.0% (0/1)	0	0	✓ 0.00%	
0800 0780 CMP R0, #3	N/A	✓ 0.0% (0/1)	0	0	✓ 0.00%	
0800 077E BEQ 0x080007C2	N/A	✓ 0.0% (0/1)	0	0	✓ 0.00%	
0800 077C CMP R0, #2	N/A	✓ 0.0% (0/1)	0	0	✓ 0.00%	
0800 077A BEQ 0x080007AE	N/A	✓ 0.0% (0/1)	0	0	✓ 0.00%	
0800 0778 CMP R0, #1	N/A	✓ 0.0% (0/1)	0	0	✓ 0.00%	
C 116: break;	100.0% (1/1)	✓ 100.0% (2/2)	712 545	1 425 090	✓ 1.15%	
C 115: bsp_DelayUS(1);	/	✓ 100.0% (2/2)	712 545	1 425 090	✓ 1.15%	
C 154: }	100.0% (1/1)	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
C 111: __NOP();	100.0% (1/1)	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
C 107: {	100.0% (1/1)	✓ 100.0% (1/1)	712 545	712 545	✓ 0.57%	
C 152: break;	0.0% (0/1)	✓ 0.0% (0/1)	0	0	✓ 0.00%	
C 149: break;	0.0% (0/1)	✓ 0.0% (0/1)	0	0	✓ 0.00%	
C 148: work_status = IDLE; /* 进	0.0% (0/1)	✓ 0.0% (0/3)	0	0	✓ 0.00%	
C 146: crc_error_times++;	0.0% (0/1)	✓ 0.0% (0/5)	0	0	✓ 0.00%	
C 142: crc_ok_times++;	0.0% (0/1)	✓ 0.0% (0/6)	0	0	✓ 0.00%	
C 140: Compute_Freq(spi_rx_buff[1],	0.0% (0/1)	✓ 0.0% (0/7)	0	0	✓ 0.00%	
C 138: if(crc_cal == spi_rx_buff[SP	0.0% (0/1)	✓ 0.0% (0/6)	0	0	✓ 0.00%	
C 136: crc_cal = crc_cal+spi_rx_buf	0.0% (0/1)	✓ 0.0% (0/9)	0	0	✓ 0.00%	

通过鼠标选中Exclude All NOP可以实现排除掉空指令（NOP）的执行、统计状况

The screenshot shows the 'Code Profile' window with the following data:

Function	Source Coverage	Inst. Coverage	Run Count	Fetch Count	Load
* OS_Idle	100.0% (1/1)	100.0% (1/1)	451 258 951	451 258 951	
* SysTick_Handler	50.0% (4/8)	84.8% (93/110)	5 399	448 438	
154: OS_INT_LeaveNestable();	0.0% (0/1)	86.8% (64/74)	5 399	275 670	
0800 0C40 MSR BASEPRI, R0	N/A	100.0% (1/1)	5 399	5 399	
0800 0C3E MOVS R0, #0	N/A	100.0% (1/1)	5 399	5 399	
0800 0C3C STRB R2, [R3]	N/A	100.0% (1/1)	5 399	5 399	
0800 0C3A LDR R3, =OS_Global ;	N/A	100.0% (1/1)	5 399	5 399	
0800 0C38 UXTB R2, R3	N/A	100.0% (1/1)			
0800 0C36 SUBS R3, #1	N/A	100.0% (1/1)			
0800 0C34 UXTB R3, R3	N/A	100.0% (1/1)			
0800 0C32 LDRB R3, [R3]	N/A	100.0% (1/1)			
0800 0C30 LDR R3, =OS_Global ;	N/A	100.0% (1/1)			
0800 0BE8 BEQ 0x08000C10	N/A	100.0% (1/1)			
0800 0BE6 CMP R3, #0	N/A	100.0% (1/1)			
0800 0BE4 LDR R3, [R3, #4]	N/A	100.0% (1/1)			
0800 0BE2 LDR R3, =OS_Global ;	N/A	100.0% (1/1)			
0800 0BE0 BNE 0x08000C10	N/A	0.0% (0/1)			
0800 0BDE CMP R3, #1	N/A	100.0% (1/1)			
0800 0BDC UXTB R3, R3	N/A	100.0% (1/1)			
0800 0BDA LDRB R3, [R3]	N/A	100.0% (1/1)			
0800 0BD8 LDR R3, =OS_Global ;	N/A	100.0% (1/1)			
0800 0BD6 NOP					
0800 0BD4 NOP					
0800 0BD2 CPSIE i	N/A	100.0% (1/1)			
0800 0BCE MSR BASEPRI, R0	N/A	100.0% (1/1)			
0800 0BCC CPSID i	N/A	100.0% (1/1)			

The context menu is open, showing the following options:

- Show Disassembly (Ctrl+D)
- Exclude from Load & Code Coverage (Del)
- Excluded From Code Coverage
- Excluded From Load
- Exclude All NOP Instructions (highlighted)
- Exclude...
- Include...
- Remove All Filters (Alt+Del)
- Show Filters...
- Collapse All (Alt+-)
- Reset Execution Counters (Ctrl+R)
- Execution Counters in Source (checked)
- Execution Counters in Disassembly
- Group By Files

也可以选择性的Export需要的跟踪记录

Code Profile						
Function	Source Coverage	Inst. Coverage	Run Count	Fetch Count	Load	
*	*	*	*	*	*	
+ f OS_Idle	100.0% (1/1)	✓ 100.0% (1/1)	948 240 840	13 948 240 840		
+ f SysTick_Handler	50.0% (4/8)	✓ 85.0% (96/113)	2 440 370	202 697 131		
+ f _SendPacket	8.1% (3/3)				21.57%	
+ f SEGGER_SYSVIEW_RecordEnterI	87.5% (7/8)				16.76%	
+ f OS_TICK_Handle	N/A				11.73%	
- f OS_ARM_ISRCheckPrio	N/A				10.05%	
- [NoSource]	N/A				10.05%	
0800 2A72 B 0x08002A6	N/A				0.84%	
0800 2A70 LDRB R3, [R	N/A				0.84%	
0800 2A6E LDR R3, =0x	N/A				0.84%	
0800 2A6C BEQ 0x08002	N/A				0.84%	
0800 2A6A CMP R0, #14	N/A				0.84%	
0800 2A68 BX LR	N/A				0.84%	
0800 2A66 MOVGE R0, #	N/A				0.84%	
0800 2A64 MOVL T R0, #	N/A				0.84%	
0800 2A62 ITE LT	N/A				0.84%	
0800 2A60 CMP R3, #0x	N/A				0.84%	
0800 2A5A BLE 0x08002	N/A				0.84%	
0800 2A58 CMP R0, #15	N/A				0.84%	
0800 2A76 BX LR	N/A				0.00%	
0800 2A74 MOVS R0, #4	N/A				0.00%	
0800 2A5E LDRB R3, [R	N/A				0.00%	
0800 2A5C LDR R3, =0x	N/A				0.00%	
+ f JLINKMEM_Process	21.1% (4/1				7.54%	
+ f SEGGER_SYSVIEW_RecordExitIS	80.0% (4/5				7.39%	
+ f _PreparePacket	100.0% (3/3				6.29%	
+ f OS_GetCPUState	N/A				4.31%	
+ f OS_ChangeTask_VFP	N/A				2.97%	
+ f OS_AssertCPUNotInIllegalSta	N/A				2.58%	
+ f OS_AssertCPUInISRMode	N/A				2.51%	
+ f SEGGER_SYSVIEW_OnTaskStopRe	77.8% (7/9				1.07%	
+ f SEGGER_SYSVIEW_OnTaskStartR	100.0% (8/8				0.89%	
+ f SEGGER_SYSVIEW_OnTaskStartE	100.0% (8/8)	✓ 100.0% (54/54)	61 010	5 185 850	0.89%	
+ f PendSV_Handler	N/A	✓ 75.4% (43/57)	109 818	2 952 882	0.51%	
+ f SEGGER_SYSVIEW_RecordU32	85.7% (6/7)	✓ 98.0% (49/50)	61 010	2 489 208	0.43%	
+ f OS_TASK_Delay	N/A	✓ 57.1% (36/63)	61 010	2 379 376	0.41%	
+ f BSP_ToggleLED	83.3% (5/6)	✓ 98.1% (52/53)	61 010	2 348 885	0.40%	
+ f SEGGER_SYSVIEW_RecordEndCo	85.7% (6/7)	✓ 75.0% (36/48)	61 010	2 257 370	0.38%	

Export Code Profile Data

Report Scope

☐ Whole Application
 ☒ Selected Functions
 选择性生成跟踪记录

☒ BSP.c
☐ core_cm4.h
☒ HardFaultHandler.S
☒ JLINKMEM_Process.c
☒ OS_Error.c
☒ OS_TraceDemo.c
☐ RTOSInit_STM32F4xx.c
☐ SEGGER_HardFaultHandler.c
☐ SEGGER_RTT.c

Output Format

☒ Report
☐ CSV

CSV Format

☐ Functions
☐ Source Lines
☐ Instructions

Other Options

☐ Export File Paths

Output File

C:/Users/Ning/Ozone_CP_191206.txt

OK

Cancel

生成的记录如下所示：

C: > Users > Ning > Ozone_CP_191206.txt

1 bzone Code Profile Report

2
3 Project: C:/Users/Ning/Desktop/J-Trace_PRO_CortexM_Tutorial/SEGGER_CortexM_Trace_Reference_Board.jdebug
4 Application: C:/Users/Ning/Desktop/J-Trace_PRO_CortexM_Tutorial/Output/Debug/Start_STM32F407_Trace_Example.elf
5 Date: 06 12? 2019
6
7

8 Code Coverage Summary

9	Module/Function	Source Lines			Instructions		
10							
11	-----+-----+-----						
12	BSP.c						
13	BSP_Init	9 /	9	100.0%	89 /	89	100.0%
14	BSP_ToggleLED	5 /	6	83.3%	52 /	53	98.1%
15	HardFaultHandler.S						
16	HardFault_Handler	0 /	6	0.0%	0 /	6	0.0%
17	JLINKMEM_Process.c						
18	_DropTxData	0 /	4	0.0%	0 /	14	0.0%
19	_FillTxBuf	0 /	20	0.0%	0 /	64	0.0%
20	_Init	14 /	14	100.0%	59 /	59	100.0%
21	_LockTxBuf	0 /	9	0.0%	0 /	25	0.0%
22	_Receive	0 /	9	0.0%	0 /	56	0.0%
23	_RetriggerTimer	0 /	2	0.0%	0 /	5	0.0%
24	_Send	0 /	11	0.0%	0 /	36	0.0%
25	_StopTimer	0 /	2	0.0%	0 /	6	0.0%
26	_UnlockTxBuf	0 /	2	0.0%	0 /	6	0.0%
27	JLINKMEM_Process	4 /	19	21.1%	20 /	62	32.3%
28	JLINKMEM_SendChar	0 /	21	0.0%	0 /	63	0.0%
29	JLINKMEM_SetpfGetNextChar	3 /	3	100.0%	8 /	8	100.0%
30	JLINKMEM_SetpfOnRx	3 /	3	100.0%	8 /	8	100.0%
31	JLINKMEM_SetpfOnTx	3 /	3	100.0%	8 /	8	100.0%
32	OS_Error.c						
33	OS_Error	0 /	7	0.0%	0 /	23	0.0%
34	OS_TraceDemo.c						
35	HPTask	6 /	6	100.0%	18 /	18	100.0%
36	LPTask	6 /	6	100.0%	18 /	18	100.0%
37	main	6 /	9	66.7%	47 /	61	77.0%
38	[NoSource]						
39	OS_Switch	0 /	0	0.0%	11 /	11	100.0%
40	-----+-----+-----						
41	Total	59 /	171	34.5%	338 /	699	48.4%
42							

43
44 Code Profile Summary

45	Module/Function	Run Count	Fetch Count
46			
47	-----+-----		
48	BSP.c		
49	BSP_Init	1	226
50	BSP_ToggleLED	61 010	2 348 885
51	HardFaultHandler.S		
52	HardFault_Handler	0	0
53	JLINKMEM_Process.c		
54	_DropTxData	0	0

55	_FillTxBuf	0	0
56	_Init	1	59
57	_LockTxBuf	0	0
58	_Receive	0	0
59	_RetriggerTimer	0	0
60	_Send	0	0
61	_StopTimer	0	0
62	_UnlockTxBuf	0	0
63	JLINKMEM_Process	2 440 370	43 926 664
64	JLINKMEM_SendChar	0	0
65	JLINKMEM_SetpfGetNextChar	1	8
66	JLINKMEM_SetpfOnRx	1	8
67	JLINKMEM_SetpfOnTx	1	8
68	OS_Error.c		
69	OS_Error	0	0
70	OS_TraceDemo.c		
71	HPTask	1	684 776
72	LPTask	1	171 194
73	main	1	50
74	[NoSource]		
75	OS_Switch	61 010	671 110
76			
77	Total	2 562 398	47 802 988
78			

这篇帖子又写的很长.....看来只能下篇帖子写特殊功能了

本主题由 eric2013 于 2019-12-7 10:08 添加图章 版主推荐

分类: debug

标签: debug

好文要顶

关注我

收藏该文

微信分享



ramlife

粉丝 - 15 关注 - 1

会员号: 2432 (终身会员PLUS)

+加关注

0

0

[升级成为会员](#)

posted on 2020-01-21 17:24 ramlife 阅读(3288) 评论(0) 收藏 举报